

ICPC Cheat Sheet

Cuaderno de Algoritmos para Competencias

Equipo: [DBC++]

Universidad: [Instituto Tecnológico de Zacatepec]

C++17 • Todos los algoritmos compilables • Probados en jueces en línea

Contents

1	Plantilla Base y E/S Rápida	3
1.1	Plantilla completa	3
1.2	E/S rápida con scanf/printf	3
1.3	Uso de <code>__int128</code>	3
1.4	Comparadores personalizados	3
1.5	Compresión de coordenadas	3
1.6	Trucos con <code>next_permutation</code>	3
2	Estructuras de Datos	4
2.1	Sparse Table	4
2.2	Next/Previous Greater/Smaller Element	4
2.3	Trucos con Stack y Queue	5
2.4	Segment Tree (actualización puntual)	5
2.5	Segment Tree con Lazy Propagation	5
2.6	BIT / Fenwick Tree	6
2.7	DSU / Union-Find	6
2.8	Trie	6
3	Grafos	8
3.1	BFS (Breadth-First Search)	8
3.2	0-1 BFS	8
3.3	DFS (Depth-First Search)	8
3.4	Dijkstra	8
3.5	Bellman-Ford	9
3.6	Floyd-Warshall	9
3.7	Orden Topológico	9
3.8	Kruskal (Árbol de Expansión Mínima)	9
3.9	Prim (Árbol de Expansión Mínima)	10
3.10	Componentes Fuertemente Conexas (Kosaraju)	10
3.11	Puentes y Puntos de Articulación	10
3.12	LCA (Ancestro Común Más Bajo — Binary Lifting)	10
3.13	Heavy-Light Decomposition (HLD)	11
4	Flujo en Redes y Matching	12
4.1	Max Flow (Dinic)	12
4.2	Max Bipartite Matching (Hopcroft-Karp)	12
5	Flujo en Redes II	13
5.1	2-SAT	13
5.2	Min-Cost Max-Flow (Dijkstra + Potencial)	13
6	Teoría de Números	14
6.1	Criba de Eratóstenes	14
6.2	Factorización en Primos	14
6.3	Exponenciación Modular Rápida	14
6.4	GCD y LCM	14
6.5	Euclides Extendido	14
6.6	Inverso Modular	14
6.7	Combinatoria ($\binom{n}{r} \bmod p$)	15
6.8	Teorema Chino del Resto (CRT)	15
6.9	SPF — Factorización en $O(\log n)$	15
6.10	Función φ de Euler (Totient)	15
6.11	Miller-Rabin (Determinístico 64-bit)	15
6.12	Pollard's Rho (Factorización grandes)	16

7 Teoría de Juegos	17
7.1 Juego de Nim y Teorema de Sprague-Grundy	17
7.1.1 Teorema de Sprague-Grundy	17
7.2 Juegos Clásicos	17
7.2.1 Nim con restricción (Bounded Nim)	17
7.2.2 Staircase Nim	17
7.2.3 Juego de Euclides	17
7.3 Referencia Rápida	17
8 Cadenas (Strings)	18
8.1 KMP (Knuth-Morris-Pratt)	18
8.2 Z-Algorithm	18
8.3 Hashing de Cadenas (Polynomial Rolling)	18
8.4 Suffix Array (Básico)	18
8.5 Manacher (Palíndromos en $O(n)$)	19
9 Programación Dinámica	20
9.1 Mochila 0/1 (Knapsack)	20
9.2 Subsecuencia Común Más Larga (LCS)	20
9.3 Subsecuencia Creciente Más Larga (LIS)	20
9.4 Digit DP (Plantilla)	20
10 Geometría	22
10.1 Punto, Producto Cruz y Producto Punto	22
10.2 Envoltente Convexa (Convex Hull — Andrew's Monotone Chain)	22
10.3 Intersección de Líneas	22
11 Técnicas de Búsqueda	23
11.1 Búsqueda Binaria sobre la Respuesta	23
11.2 Dos Punteros (Two Pointers)	23
11.3 Ventana Deslizante (Sliding Window)	23
11.4 Stack Monotónico (NGE/NSE/PGE/PSE)	23
11.5 Mo's Algorithm	23
12 Miscelánea	25
12.1 Recorrido del Caballo (Warnsdorff)	25
12.2 Operaciones de Bits Útiles	25
12.3 Ordered Set (pb_ds)	25
12.4 Trucos STL Frecuentes	25
12.5 Sumas de Prefijo 2D	25
13 Álgebra Lineal	26
13.1 Eliminación Gaussiana (módulo)	26
13.2 Exponenciación de Matrices	26
13.3 FFT (Fast Fourier Transform)	26
13.4 NTT (Number Theoretic Transform)	26
14 Scripts Útiles (Compile y Probar)	28
14.1 Compilación Estándar (C++17)	28
14.2 Compilación y Ejecución Directa (CMD/Bash)	28
14.3 Redirección de I/O desde el Código	28
15 Patrones Rápidos	28

Plantilla Base y E/S Rápida

Plantilla completa

Tiempo: — **Espacio:** —

Cuándo usarla: Al inicio de cada problema. Incluye tipos, macros y E/S rápida.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 using lli = long long int;
6 using pii = pair<int,int>;
7 using pll = pair<ll,ll>;
8 using vi = vector<int>;
9 using vll = vector<ll>;
10
11 #define fast ios::sync_with_stdio(false); cin.tie(nullptr)
12 #define all(x) (x).begin(),(x).end()
13 #define sz(x) (int)(x).size()
14 #define pb push_back
15 #define fi first
16 #define se second
17
18 const int MOD = 1e9 + 7;
19 const ll INF = 1e18;
20
21 void solve(){
22
23 }
24
25 int main(){
26     fast;
27     int t = 1;
28     // cin >> t;
29     while(t--) solve();
30     return 0;
31 }
```

E/S rápida con scanf/printf

Tiempo: — **Espacio:** —

Cuándo usarla: Cuando cin/cout son demasiado lentos (datos masivos > 10^6 líneas).

```
1 // Added -- not in notebook
2 #include <cstdio>
3 // Leer entero
4 int x; scanf("%d", &x);
5 // Leer long long
6 long long y; scanf("%lld", &y);
7 // Leer string sin espacios
8 char s[200001]; scanf("%s", s);
9 // Imprimir
10 printf("%d %lld\n", x, y);
```

Uso de __int128

Tiempo: — **Espacio:** —

Cuándo usarlo: Cuando se necesita precisión mayor que long long ($> 9.2 \times 10^{18}$), típicamente en multiplicaciones intermedias.

```
1 // Added -- not in notebook
2 // No se puede usar cin/cout directamente.
3 // Funcion auxiliar para imprimir:
4 void print128(__int128 x){
```

```
5     if(x < 0){ putchar('-'); x = -x; }
6     if(x > 9) print128(x / 10);
7     putchar('0' + (int)(x % 10));
8 }
9 // Uso tipico: evitar overflow en a*b mod m
10 long long mulmod(long long a, long long b, long long m){
11     return ((__int128)a * b) % m;
12 }
```

Comparadores personalizados

Tiempo: — **Espacio:** —

Cuándo usarlos: Ordenar por criterios no estándar, priority_queue con min-heap, sets con orden custom.

```
1 // Added -- not in notebook
2 // Lambda para sort
3 sort(all(v), [](const pii &a, const pii &b){
4     if(a.se != b.se) return a.se < b.se;
5     return a.fi < b.fi;
6 });
7 // Struct para priority_queue (min-heap)
8 struct Cmp {
9     bool operator()(const pii &a, const pii &b){
10         return a.se > b.se; // menor primero
11     }
12 };
13 priority_queue<pii, vector<pii>, Cmp> pq;
14 // Min-heap rapido con greater
15 priority_queue<int, vector<int>, greater<int>> minpq;
```

Compresión de coordenadas

Tiempo: $O(n \log n)$ **Espacio:** $O(n)$

Cuándo usarla: Valores grandes ($\leq 10^9$) pero pocos elementos distintos ($\leq 10^5$). Permite usar arreglos indexados.

```
1 // Added -- not in notebook
2 // Dado vector<int> a de tamaño n:
3 vi sorted_a = a;
4 sort(all(sorted_a));
5 sorted_a.erase(unique(all(sorted_a)), sorted_a.end());
6 // Comprimir: a[i] -> índice en sorted_a
7 for(int &x : a)
8     x = lower_bound(all(sorted_a), x) - sorted_a.begin();
9 // Ahora a[i] esta en [0, sorted_a.size()-1]
```

Trucos con next_permutation

Tiempo: $O(n!)$ total **Espacio:** $O(n)$

Cuándo usarlo: Fuerza bruta sobre permutaciones ($n \leq 10$).

```
1 // Added -- not in notebook
2 vi a = {1, 2, 3, 4};
3 sort(all(a)); // IMPORTANTE: empezar ordenado
4 do {
5     // procesar permutacion actual a[]
6 } while(next_permutation(all(a)));
7
8 // Permutaciones de un string
9 string s = "abc";
10 sort(all(s));
11 do {
12     // procesar s
13 } while(next_permutation(all(s)));
```

Estructuras de Datos

Sparse Table

Tiempo: Construcción $O(n \log n)$, consulta $O(1)$ **Espacio:** $O(n \log n)$

Cuándo usarla: Consultas RMQ (mínimo/máximo en rango) estáticas, sin actualizaciones.

Explicación: Precalcula el mínimo/máximo para intervalos de longitud 2^k . Para consultar $[l, r]$, toma el mayor k tal que $2^k \leq len$ y combina los dos intervalos superpuestos. Los índices son 0-based.

```
1 // Del notebook: clasicos/ED/Sparse-Table.cpp
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 vector<vector<int>> pre_min(vector<int> &input, int n){
6     vector<vector<int>> sparse(n, vector<int>(1+(int)log2(n)));
7     int i, j;
8     for(i = 0; i < n; i++) sparse[i][0] = i;
9     for(j = 1; (1<<j) <= n; j++){
10         for(i = 0; i + (1<<j) - 1 < n; i++){
11             if(input[sparse[i][j-1]] < input[sparse[i+(1<<(j-1))][j-1]])
12                 sparse[i][j] = sparse[i][j-1];
13             else
14                 sparse[i][j] = sparse[i+(1<<(j-1))][j-1];
15         }
16     }
17     return sparse;
18 }
19
20 vector<vector<int>> pre_max(vector<int> &input, int n){
21     vector<vector<int>> sparse(n, vector<int>(1+(int)log2(n)));
22     int i, j;
23     for(i = 0; i < n; i++) sparse[i][0] = i;
24     for(j = 1; (1<<j) <= n; j++){
25         for(i = 0; i + (1<<j) - 1 < n; i++){
26             if(input[sparse[i][j-1]] > input[sparse[i+(1<<(j-1))][j-1]])
27                 sparse[i][j] = sparse[i][j-1];
28             else
29                 sparse[i][j] = sparse[i+(1<<(j-1))][j-1];
30         }
31     }
32     return sparse;
33 }
34
35 int query(int l, int r,
36           const vector<vector<int>>& sparse,
37           const vector<int>& input){
38     int len = r - l + 1;
39     int k = (int)log2(len);
40     int leftI = sparse[l][k];
41     int rightI = sparse[r - (1<<k) + 1][k];
42     return min(input[leftI], input[rightI]);
43 }
44 // --- USO ---
45 // vector<int> a = {3,1,4,1,5}; int n = a.size();
46 // auto sp = pre_min(a, n); // preprocesar mínimos
47 // int mn = query(1, 3, sp, a); // mínimo en a[1..3] (0-based)
48 // pre_min/pre_max: recibe arreglo y n, retorna tabla.
49 // query(l, r, tabla, arreglo): retorna min/max en [l,r].
50 // Índices 0-based. NO soporta actualizaciones.
```

Aplicado en: **Maximum in a Window** (practica-cpc) — Sparse Table para máximo en ventana deslizante con consultas $O(1)$.

```
1 // practica-cpc/maximun-in-a-window.cpp (extracto)
2 int main(){
3     int n, k; cin >> n >> k;
4     vector<int> a(n);
5     for(int &x : a) cin >> x;
6     auto sparse = pre_max(a, n); // pre_max
7     int i = 0, j = k - 1;
8     while(j < n){
9         cout << query(i, j, sparse, a) << ' ';
10        i++; j++;
11    }
12 }
```

Next/Previous Greater/Smaller Element

Tiempo: $O(n)$ **Espacio:** $O(n)$

Cuándo usarlo: Encontrar el siguiente/anterior elemento mayor/menor para cada posición. Base de histograma, imbalance, stock span.

Explicación: Usan un stack monotónico. Cada función devuelve un vector de **índices** (no valores). Por ejemplo, 'nge(a)[i]' es el índice del primer elemento a la derecha de 'i' que es estrictamente mayor que 'a[i]'; si no existe, devuelve 'n'. Las versiones 'pge' (previous greater) devuelven índice del anterior mayor a la izquierda, o '-1' si no existe. Para obtener el valor, usa 'a[idx]'. Útil para calcular áreas en histograma.

```
1 // Del notebook: clasicos/ED/nse-nge-pse-pge.cpp
2 // Version con índices (mas practica)
3 vector<int> nge(vector<int> a){
4     stack<int> s;
5     int n = a.size();
6     vector<int> sol(n);
7     for(int i = n-1; i >= 0; i--){
8         while(!s.empty() && a[i] >= a[s.top()])
9             s.pop();
10        sol[i] = s.empty() ? n : s.top();
11        s.push(i);
12    }
13    return sol;
14 }
15
16 vector<int> nse(vector<int> a){
17     stack<int> s;
18     int n = a.size();
19     vector<int> sol(n);
20     for(int i = n-1; i >= 0; i--){
21         while(!s.empty() && a[i] <= a[s.top()])
22             s.pop();
23        sol[i] = s.empty() ? n : s.top();
24        s.push(i);
25    }
26    return sol;
27 }
28
29 vector<int> pge(vector<int> a){
30     stack<int> s;
31     int n = a.size();
32     vector<int> sol(n);
33     for(int i = 0; i < n; i++){
34         while(!s.empty() && a[i] > a[s.top()])
35             s.pop();
36        sol[i] = s.empty() ? -1 : s.top();
37        s.push(i);
38    }
39    return sol;
40 }
41
42 vector<int> pse(vector<int> a){
43     stack<int> s;
44     int n = a.size();
45     vector<int> sol(n);
46     for(int i = 0; i < n; i++){
47         while(!s.empty() && a[i] < a[s.top()])
48             s.pop();
49        sol[i] = s.empty() ? -1 : s.top();
50        s.push(i);
51    }
52    return sol;
53 }
54 // --- USO ---
55 // Cada funcion recibe vector<int> y retorna vector<int>
56 // de INDICES (no valores). Ejemplo: a = {3,1,4,1,5}
57 // nge(a) -> para cada i, indice del proximo mayor a la
58 // derecha. Si no hay, retorna n.
59 // nse(a) -> proximo menor a la derecha. Si no hay, n.
60 // pge(a) -> previo mayor a la izquierda. Si no hay, -1.
61 // pse(a) -> previo menor a la izquierda. Si no hay, -1.
62 // auto ng = nge(a); // ng[0]=2 porque a[2]=4 > a[0]=3
```

Aplicado en: **Histograma (SPOJ)** — Área máxima del

rectángulo en histograma usando PSE y NSE.

```
1 // vjudge/histograma.cpp (extracto)
2 long long largestRectangleArea(vector<ll>& h){
3     int n = h.size();
4     vector<ll> NSE = nse(h);
5     vector<ll> PSE = pse(h);
6     long long sol = 0;
7     for(int j = 0; j < n; j++){
8         sol = max(sol, h[j]*(NSE[j]-PSE[j]-1));
9     }
10 }
```

Aplicado en: **Imbalanced (CF)** — Suma de (max - min) de todos los subarreglos usando contribución de cada elemento.

```
1 // practica-cpc/imbalanced.cpp (extracto)
2 ll total = 0;
3 for(int i = 0; i < n; i++){
4     ll rmax = 1LL*(i-pge[i])*(nge[i]-i);
5     ll rmin = 1LL*(i-pse[i])*(nse[i]-i);
6     total += rmax*a[i] - rmin*a[i];
7 }
```

Trucos con Stack y Queue

Tiempo: $O(n)$ **Espacio:** $O(n)$

Cuándo usarlos: Matching de paréntesis, secuencias regulares, evaluación de expresiones.

```
1 // Del notebook: practica-cpc (bracket problems)
2 // Contar parentesis sobrantes (no matcheados)
3 int unmatchedBrackets(string &st){
4     stack<char> s;
5     for(char &x : st){
6         if(x == '(') s.push(x);
7         else {
8             if(!s.empty() && s.top() == '(') s.pop();
9             else s.push(x);
10        }
11    }
12    return s.size();
13 }
14
15 // Longest Regular Bracket Sequence + conteo
16 pair<int,int> longestRegular(string &st){
17     stack<int> s;
18     s.push(-1);
19     int mx = 0, count = 1;
20     for(int i = 0; i < (int)st.length(); i++){
21         if(st[i] == '('){
22             s.push(i);
23         } else {
24             s.pop();
25             if(s.empty()){
26                 s.push(i);
27             } else {
28                 int k = i - s.top();
29                 if(k > mx){ mx = k; count = 1; }
30                 else if(k == mx) count++;
31             }
32         }
33     }
34     return {mx, count};
35 }
```

Segment Tree (actualización puntual)

Tiempo: Construcción $O(n)$, consulta/actualización $O(\log n)$ **Espacio:** $O(4n)$

Cuándo usarlo: Consultas de rango (suma, mín, máx) con actualizaciones puntuales dinámicas.

Explicación: El árbol se construye recursivamente. Cada nodo almacena la suma (u otra operación asociativa) de un intervalo. Para actualizar una posición, se cambia el valor en las hojas y se propaga hacia arriba. Para consultar un rango, se recorren los nodos que cubren exactamente el intervalo pedido.

```
1 // Added -- not in notebook
2 struct SegTree {
```

```
3     int n;
4     vector<ll> tree;
5
6     SegTree(int n) : n(n), tree(4*n, 0) {}
7
8     void build(vector<ll> &a, int v, int tl, int tr){
9         if(tl == tr){
10             tree[v] = a[tl];
11             return;
12         }
13         int tm = (tl + tr) / 2;
14         build(a, 2*v, tl, tm);
15         build(a, 2*v+1, tm+1, tr);
16         tree[v] = tree[2*v] + tree[2*v+1];
17     }
18
19     void build(vector<ll> &a){ build(a, 1, 0, n-1); }
20
21     void update(int v, int tl, int tr, int pos, ll val){
22         if(tl == tr){
23             tree[v] = val;
24             return;
25         }
26         int tm = (tl + tr) / 2;
27         if(pos <= tm) update(2*v, tl, tm, pos, val);
28         else update(2*v+1, tm+1, tr, pos, val);
29         tree[v] = tree[2*v] + tree[2*v+1];
30     }
31
32     void update(int pos, ll val){
33         update(1, 0, n-1, pos, val);
34     }
35
36     ll query(int v, int tl, int tr, int l, int r){
37         if(l > r) return 0;
38         if(l == tl && r == tr) return tree[v];
39         int tm = (tl + tr) / 2;
40         return query(2*v, tl, tm, l, min(r, tm))
41             + query(2*v+1, tm+1, tr, max(l, tm+1), r);
42     }
43
44     ll query(int l, int r){
45         return query(1, 0, n-1, l, r);
46     }
47 };
48
49 // --- USO ---
50 // SegTree st(n); // crear con n elementos
51 // st.build(a); // construir desde vector a (0-based)
52 // st.update(i, val); // a[i] = val (REEMPLAZA, no suma)
53 // ll s = st.query(l, r); // suma de a[l..r] (0-based, inclusive)
54 // Para min/max: cambiar + por min/max en build/update/query,
55 // y retornar INF en vez de 0 en caso base de query.
```

Segment Tree con Lazy Propagation

Tiempo: Consulta/actualización de rango $O(\log n)$ **Espacio:** $O(4n)$

Cuándo usarlo: Actualizaciones de rango (sumar valor a $[l, r]$, asignar valor a $[l, r]$) con consultas de rango.

Explicación: A diferencia del segment tree normal, aquí las actualizaciones pueden afectar rangos completos. Para no actualizar cada hoja, se usa un vector 'lazy' que almacena la operación pendiente sobre un nodo. Al recorrer, se "empuja" ('push') la operación a los hijos antes de continuar.

```
1 // Added -- not in notebook
2 struct LazySegTree {
3     int n;
4     vector<ll> tree, lazy;
5
6     LazySegTree(int n) : n(n), tree(4*n, 0), lazy(4*n, 0) {}
7
8     void build(vector<ll> &a, int v, int tl, int tr){
9         if(tl == tr){ tree[v] = a[tl]; return; }
10        int tm = (tl+tr)/2;
11        build(a, 2*v, tl, tm);
12        build(a, 2*v+1, tm+1, tr);
13        tree[v] = tree[2*v] + tree[2*v+1];
14    }
15
16    void build(vector<ll> &a){ build(a, 1, 0, n-1); }
17
18    void push(int v, int tl, int tr){
```

```

19     if(lazy[v] != 0){
20         int tm = (tl+tr)/2;
21         apply(2*v, tl, tm, lazy[v]);
22         apply(2*v+1, tm+1, tr, lazy[v]);
23         lazy[v] = 0;
24     }
25 }
26
27 void apply(int v, int tl, int tr, ll val){
28     tree[v] += val * (tr - tl + 1);
29     lazy[v] += val;
30 }
31
32 void update(int v, int tl, int tr, int l, int r, ll val){
33     if(l > r) return;
34     if(l == tl && r == tr){
35         apply(v, tl, tr, val);
36         return;
37     }
38     push(v, tl, tr);
39     int tm = (tl+tr)/2;
40     update(2*v, tl, tm, l, min(r, tm), val);
41     update(2*v+1, tm+1, tr, max(l, tm+1), r, val);
42     tree[v] = tree[2*v] + tree[2*v+1];
43 }
44
45 void update(int l, int r, ll val){
46     update(1, 0, n-1, l, r, val);
47 }
48
49 ll query(int v, int tl, int tr, int l, int r){
50     if(l > r) return 0;
51     if(l == tl && r == tr) return tree[v];
52     push(v, tl, tr);
53     int tm = (tl+tr)/2;
54     return query(2*v, tl, tm, l, min(r, tm))
55         + query(2*v+1, tm+1, tr, max(l, tm+1), r);
56 }
57
58 ll query(int l, int r){
59     return query(1, 0, n-1, l, r);
60 }
61 };
62 // --- USO ---
63 // LazySegTree st(n); // crear con n elementos
64 // st.build(a); // construir desde vector a (0-based)
65 // st.update(l, r, val); // SUMAR val a todos a[l..r]
66 // ll s = st.query(l, r); // suma de a[l..r] (0-based)
67 // A diferencia del SegTree normal, aquí update es de RANGO:
68 // st.update(2, 5, 10) suma 10 a a[2], a[3], a[4], a[5].
69 // Para asignación en rango: cambiar apply() para que
70 // tree[v] = val*(tr-tl+1) y lazy[v] = val (no +=).

```

BIT / Fenwick Tree

Tiempo: Actualización $O(\log n)$, consulta prefijo $O(\log n)$

Espacio: $O(n)$

Cuándo usarlo: Sumas de prefijo con actualizaciones puntuales. Más simple y rápido que Segment Tree cuando basta con sumas.

Explicación: BIT almacena sumas acumuladas en índices estratégicos. La operación 'update(i, delta)' suma 'delta' al elemento en posición 'i' (índices 1-based). Para obtener la suma del prefijo '[1..i]', se usa 'query(i)'. Para sumar en rango '[l..r]' se calcula 'query(r) - query(l-1)'. Para asignar un valor en lugar de sumar, se debe calcular la diferencia con el valor anterior: 'update(i, newVal - oldVal)'.

```

1 // Added -- not in notebook
2 struct BIT {
3     int n;
4     vector<ll> tree;
5
6     BIT(int n) : n(n), tree(n+1, 0) {}
7
8     void update(int i, ll delta){
9         for(; i <= n; i += i & (-i))
10             tree[i] += delta;
11     }
12
13     ll query(int i){
14         ll s = 0;

```

```

15         for(; i > 0; i -= i & (-i))
16             s += tree[i];
17         return s;
18     }
19
20     ll query(int l, int r){
21         return query(r) - query(l-1);
22     }
23 };
24 // --- USO ---
25 // BIT bit(n); // crear, indices 1 a n
26 // bit.update(i, delta); // a[i] += delta (SUMA delta,
27 // no reemplaza). delta puede ser negativo.
28 // Ej: a[3] era 5, bit.update(3, 10) -> a[3] ahora vale 15
29 // Para setear a[i]=val: bit.update(i, val - a[i])
30 // bit.query(i); // suma de a[1..i] (prefijo)
31 // bit.query(l, r); // suma de a[l..r]
32 // IMPORTANTE: indexación 1-based (i va de 1 a n).
33 // Construir desde arreglo a[1..n]:
34 // for(int i=1; i<=n; i++) bit.update(i, a[i]);

```

DSU / Union-Find

Tiempo: $O(\alpha(n))$ amortizado por operación **Espacio:** $O(n)$

Cuándo usarlo: Componentes conexas dinámicas, Kruskal, consultas de conectividad.

Explicación: Mantiene particiones de un conjunto. 'find(x)' devuelve el representante de la componente de 'x'. 'unite(a,b)' une las componentes de 'a' y 'b', retorna 'true' si estaban separadas. 'connected(a,b)' comprueba si están en la misma componente. 'components' es el número actual de componentes.

```

1 // Added -- not in notebook
2 struct DSU {
3     vector<int> parent, rank_;
4     int components;
5
6     DSU(int n) : parent(n), rank_(n, 0), components(n){
7         iota(parent.begin(), parent.end(), 0);
8     }
9
10    int find(int x){
11        if(parent[x] != x)
12            parent[x] = find(parent[x]); // path compression
13        return parent[x];
14    }
15
16    bool unite(int a, int b){
17        a = find(a); b = find(b);
18        if(a == b) return false;
19        if(rank_[a] < rank_[b]) swap(a, b);
20        parent[b] = a;
21        if(rank_[a] == rank_[b]) rank_[a]++;
22        components--;
23        return true;
24    }
25
26    bool connected(int a, int b){
27        return find(a) == find(b);
28    }
29 };
30 // --- USO ---
31 // DSU dsu(n); // n nodos, 0-indexed [0, n-1]
32 // dsu.unite(a, b); // unir componentes de a y b.
33 // Retorna true si estaban separados, false si ya estaban juntos
34 // dsu.connected(a, b); // true si a y b están en la misma
35 // componente.
36 // dsu.find(a); // retorna el representante de la
37 // componente de a.
38 // dsu.components; // numero de componentes conexas actual.

```

Trie

Tiempo: Inserción/búsqueda $O(|s|)$ **Espacio:** $O(\text{total caracteres} \times \Sigma)$

Cuándo usarlo: Prefijos comunes, autocompletado, XOR máximo, diccionarios.

Explicación: Estructura de árbol para almacenar cadenas. Cada nodo tiene un arreglo de hijos (tamaño del alfabeto). 'insert(s)' agrega la cadena. 'search(s)' comprueba si 's' está como palabra completa. 'startsWith(prefix)' verifica si hay alguna palabra que comience con 'prefix'.

```

1 // Added -- not in notebook
2 struct Trie {
3     struct Node {
4         int children[26];
5         bool isEnd;
6         Node(){ memset(children, -1, sizeof(children)); isEnd =
            false; }
7     };
8     vector<Node> nodes;
9
10    Trie(){ nodes.emplace_back(); }
11
12    void insert(const string &s){
13        int cur = 0;
14        for(char c : s){
15            int idx = c - 'a';
16            if(nodes[cur].children[idx] == -1){
17                nodes[cur].children[idx] = nodes.size();
18                nodes.emplace_back();
19            }
20            cur = nodes[cur].children[idx];
21        }
22        nodes[cur].isEnd = true;
23    }
24

```

```

25    bool search(const string &s){
26        int cur = 0;
27        for(char c : s){
28            int idx = c - 'a';
29            if(nodes[cur].children[idx] == -1) return false;
30            cur = nodes[cur].children[idx];
31        }
32        return nodes[cur].isEnd;
33    }
34
35    bool startsWith(const string &prefix){
36        int cur = 0;
37        for(char c : prefix){
38            int idx = c - 'a';
39            if(nodes[cur].children[idx] == -1) return false;
40            cur = nodes[cur].children[idx];
41        }
42        return true;
43    }
44 };
45 // --- USO ---
46 // Trie trie;
47 // trie.insert("hola"); // agrega "hola" al diccionario
48 // trie.search("hola"); // true si "hola" fue insertada
    exacta
49 // trie.search("hol"); // false (no es palabra completa)
50 // trie.startsWith("hol"); // true (existe alguna palabra con
    prefijo "hol")
51 // Solo soporta chars 'a'-'z'. Para mayusculas o digitos,
52 // cambiar c-'a' por el offset adecuado y el 26 por el tamaño del
    alfabeto.

```


Grafos

BFS (Breadth-First Search)

Tiempo: $O(V + E)$ **Espacio:** $O(V)$

Cuándo usarlo: Camino más corto en grafos sin peso, explorar por niveles, distancia mínima.

Explicación: BFS desde un nodo fuente. Usa una cola. 'vis' evita revisitar. 'd' almacena distancias (en número de aristas). 'p' guarda el padre para reconstruir el camino.

```
1 // Del notebook: clasicos/graphs/recorridos.cpp
2 void bfs(int x, vector<vi> &adj){
3     int n = adj.size();
4     queue<int> q;
5     q.push(x);
6     vector<bool> vis(n+1, false);
7     vector<int> d(n+1, 0), p(n+1, -1);
8     vis[x] = true;
9     while(!q.empty()){
10         int u = q.front(); q.pop();
11         for(int &v : adj[u]){
12             if(!vis[v]){
13                 vis[v] = true;
14                 d[v] = d[u] + 1;
15                 p[v] = u;
16                 q.push(v);
17             }
18         }
19     }
20     // Reconstruir camino a nodoDestino:
21     // for(int i = dest; i != -1; i = p[i])
22     //     path.push_back(i);
23     // reverse(path.begin(), path.end());
24 }
```

Aplicado en: Message Route (CSES) — Camino más corto en grafo no ponderado.

```
1 // practica-cpc/message-route.cpp (extracto)
2 vector<int> dist(n+1, INT_MAX);
3 queue<int> q; q.push(1); dist[1] = 1;
4 vector<int> parent(n+1, -1);
5 while(!q.empty()){
6     int v = q.front(); q.pop();
7     for(int &u : adj[v]){
8         if(dist[u] != INT_MAX) continue;
9         dist[u] = dist[v] + 1;
10        parent[u] = v;
11        q.push(u);
12    }
13 }
14 // Reconstruir camino de 1 a n via parent[]
```

Aplicado en: Guilty Prince (LightOJ) — BFS en cuadrícula para contar celdas alcanzables.

```
1 // practica-cpc/guilty-prince.cpp (extracto)
2 int dx[] = {-1, 1, 0, 0};
3 int dy[] = {0, 0, -1, 1};
4 queue<pair<int,int>> q;
5 q.push({startX, startY});
6 vis[startX][startY] = true;
7 int ans = 1;
8 while(!q.empty()){
9     auto [x, y] = q.front(); q.pop();
10    for(int i = 0; i < 4; i++){
11        int xx = x+dx[i], yy = y+dy[i];
12        if(xx>=0 && xx<h && yy>=0 && yy<w
13            && !vis[xx][yy] && grid[xx][yy]!='.'){
14            vis[xx][yy] = true;
15            q.push({xx, yy});
16            ans++;
17        }
18    }
19 }
```

0-1 BFS

Tiempo: $O(V + E)$ **Espacio:** $O(V)$

Cuándo usarlo: Camino más corto cuando las aristas tienen peso 0 ó 1.

Explicación: Similar a Dijkstra pero con deque. Si la arista tiene peso 0, se inserta al frente; si peso 1, al final. Así se mantiene la cola ordenada por distancias.

```
1 // Del notebook: clasicos/graphs/recorridos.cpp
2 void bfs_ZO(int s, vector<vector<pii>> &adj){
3     int n = adj.size();
4     vector<int> d(n, 1e9);
5     deque<int> dq;
6     d[s] = 0;
7     dq.push_front(s);
8     while(!dq.empty()){
9         int v = dq.front(); dq.pop_front();
10        for(auto &[u, w] : adj[v]){
11            if(d[v] + w < d[u]){
12                d[u] = d[v] + w;
13                if(w == 1) dq.push_back(u);
14                else dq.push_front(u);
15            }
16        }
17    }
18 }
19 // --- USO ---
20 // adj es vector<vector<pii>> donde pii = {nodo, peso}.
21 // Peso DEBE ser 0 o 1. Usa deque en vez de queue:
22 // aristas de peso 0 van al frente, peso 1 al fondo.
23 // Ej: cuadrícula donde moverse en misma dir es gratis (0)
24 // y cambiar dir cuesta 1. d[v] = distancia minima desde s.
```

DFS (Depth-First Search)

Tiempo: $O(V + E)$ **Espacio:** $O(V)$

Cuándo usarlo: Detectar ciclos, componentes conexas, orden topológico, puentes, puntos de articulación.

```
1 // Added -- not in notebook
2 vector<bool> vis;
3 vector<int> tin, tout;
4 int timer = 0;
5
6 void dfs(int v, int p, vector<vi> &adj){
7     vis[v] = true;
8     tin[v] = timer++;
9     for(int u : adj[v]){
10        if(u == p) continue;
11        if(!vis[u]) dfs(u, v, adj);
12    }
13    tout[v] = timer++;
14 }
15
16 // Version iterativa (evita stack overflow)
17 void dfs_iter(int s, vector<vi> &adj){
18     int n = adj.size();
19     vector<bool> vis(n, false);
20     stack<int> st;
21     st.push(s);
22     while(!st.empty()){
23         int v = st.top(); st.pop();
24         if(vis[v]) continue;
25         vis[v] = true;
26         for(int u : adj[v])
27             if(!vis[u]) st.push(u);
28     }
29 }
30 // --- USO ---
31 // Version recursiva: tin[v]/tout[v] = tiempo de entrada/salida.
32 // u es ancestro de v sii tin[u] <= tin[v] && tout[v] <= tout[u].
33 // Llamar: vis.assign(n, false); tin.resize(n); tout.resize(n);
34 // timer=0; dfs(root, -1, adj);
35 // p = padre (pasar -1 para raiz). Evita volver al padre.
36 // Version iterativa: usa stack, no calcula tin/tout.
37 // Util cuando n es grande (>1e5) para evitar stack overflow.
```

Dijkstra

Tiempo: $O((V + E) \log V)$ **Espacio:** $O(V + E)$

Cuándo usarlo: Camino más corto desde una fuente, aristas con peso ≥ 0 .

Explicación: Usa una cola de prioridad (min-heap) para siempre extraer el nodo con distancia mínima actual. Relaja

sus aristas. El vector 'dist' contiene las distancias finales. Si 'dist[v] = INF', el nodo no es alcanzable.

```
1 // Added -- not in notebook
2 vector<ll> dijkstra(int s, vector<vector<pii>> &adj){
3     int n = adj.size();
4     vector<ll> dist(n, 1e18);
5     // {distancia, nodo}
6     priority_queue<pll, vector<pll>, greater<pll>> pq;
7     dist[s] = 0;
8     pq.push({0, s});
9     while(!pq.empty()){
10         auto [d, v] = pq.top(); pq.pop();
11         if(d > dist[v]) continue;
12         for(auto &[u, w] : adj[v]){
13             if(dist[v] + w < dist[u]){
14                 dist[u] = dist[v] + w;
15                 pq.push({dist[u], u});
16             }
17         }
18     }
19     return dist;
20 }
21 // --- USO ---
22 // adj: vector<vector<pii>>, pii = {nodo_vecino, peso}.
23 // auto d = dijkstra(0, adj); // distancias desde nodo 0
24 // d[v] = distancia minima de 0 a v. Si d[v]=1e18, no es
25 // alcanzable.
26 // NO funciona con pesos negativos. Para eso usar Bellman-Ford.
27 // Construir adj: adj[u].push_back({v, w}); (y adj[v].push_back({u, w}) si no dirigido)
```

Bellman-Ford

Tiempo: $O(V \cdot E)$ **Espacio:** $O(V)$

Cuándo usarlo: Camino más corto con aristas negativas. Detectar ciclos negativos.

Explicación: Relaja todas las aristas $V-1$ veces. Si después de eso aún se puede relajar, hay un ciclo negativo alcanzable desde la fuente.

```
1 // Added -- not in notebook
2 struct Edge { int u, v; ll w; };
3
4 // Retorna true si hay ciclo negativo alcanzable desde s
5 bool bellmanFord(int s, int n, vector<Edge> &edges,
6     vector<ll> &dist){
7     dist.assign(n, 1e18);
8     dist[s] = 0;
9     for(int i = 0; i < n-1; i++){
10         for(auto &[u, v, w] : edges){
11             if(dist[u] < 1e18 && dist[u]+w < dist[v]){
12                 dist[v] = dist[u] + w;
13             }
14         }
15         // Detectar ciclo negativo
16         for(auto &[u, v, w] : edges){
17             if(dist[u] < 1e18 && dist[u]+w < dist[v]){
18                 return true; // ciclo negativo
19             }
20         }
21         return false;
22     }
23 // --- USO ---
24 // vector<Edge> edges; // lista de aristas {u, v, peso}
25 // edges.push_back({0, 1, 5}); // arista de 0 a 1 con peso 5
26 // vector<ll> dist;
27 // bool cicloNeg = bellmanFord(0, n, edges, dist);
28 // dist[v] = distancia minima desde s. Si cicloNeg=true,
29 // hay ciclo negativo alcanzable (distancias no confiables).
```

Floyd-Warshall

Tiempo: $O(V^3)$ **Espacio:** $O(V^2)$

Cuándo usarlo: Distancias mínimas entre todos los pares de nodos. $V \leq 500$.

```
1 // Added -- not in notebook
2 void floydWarshall(vector<vector<ll>> &dist, int n){
3     // dist[i][j] inicializado con pesos de aristas,
4     // dist[i][i] = 0, dist[i][j] = INF si no hay arista
5     for(int k = 0; k < n; k++){
6         for(int i = 0; i < n; i++){
7             for(int j = 0; j < n; j++){
```

```
8                 if(dist[i][k] < 1e18 && dist[k][j] < 1e18)
9                     dist[i][j] = min(dist[i][j],
10                         dist[i][k]+dist[k][j]);
11             }
12         }
13     }
14     // vector<vector<ll>> dist(n, vector<ll>(n, 1e18));
15     // for(int i=0; i<n; i++) dist[i][i] = 0;
16     // Para cada arista (u,v,w): dist[u][v] = w;
17     // floydWarshall(dist, n);
18     // dist[i][j] = distancia minima de i a j.
19     // Si dist[i][j]=1e18, no hay camino. Solo para n<=500.
```

Orden Topológico

Tiempo: $O(V + E)$ **Espacio:** $O(V)$

Cuándo usarlo: Ordenar tareas con dependencias (DAG).

Detectar ciclos en grafos dirigidos.

```
1 // Added -- not in notebook
2 // Version Kahn (BFS con grado de entrada)
3 vi topoKahn(int n, vector<vi> &adj){
4     vector<int> indeg(n, 0);
5     for(int u = 0; u < n; u++){
6         for(int v : adj[u]) indeg[v]++;
7     }
8     queue<int> q;
9     for(int i = 0; i < n; i++){
10         if(indeg[i] == 0) q.push(i);
11     }
12     vi order;
13     while(!q.empty()){
14         int v = q.front(); q.pop();
15         order.push_back(v);
16         for(int u : adj[v]){
17             if(--indeg[u] == 0) q.push(u);
18         }
19     }
20     // Si order.size() < n => hay ciclo
21     return order;
22 }
23 // Version DFS
24 vi topo_order;
25 vector<int> color; // 0=blanco, 1=gris, 2=negro
26 bool has_cycle;
27 void topoDFS(int v, vector<vi> &adj){
28     color[v] = 1;
29     for(int u : adj[v]){
30         if(color[u] == 1){ has_cycle = true; return; }
31         if(color[u] == 0) topoDFS(u, adj);
32     }
33     color[v] = 2;
34     topo_order.push_back(v);
35 }
36 vi topoSort(int n, vector<vi> &adj){
37     color.assign(n, 0);
38     topo_order.clear();
39     has_cycle = false;
40     for(int i = 0; i < n; i++){
41         if(color[i] == 0) topoDFS(i, adj);
42     }
43     reverse(all(topo_order));
44     return topo_order;
45 }
46 // --- USO ---
47 // adj = lista de adyacencia DIRIGIDA. adj[u] = {vecinos de u}.
48 // vi ord = topoKahn(n, adj); // version BFS
49 // if(ord.size() < n) -> HAY CICLO, no existe orden topologico.
50 // vi ord = topoSort(n, adj); // version DFS
51 // if(has_cycle) -> HAY CICLO.
52 // ord[0] va primero, ord[n-1] va ultimo.
53 // Ej: si tarea A depende de B, arista B->A.
```

Kruskal (Árbol de Expansión Mínima)

Tiempo: $O(E \log E)$ **Espacio:** $O(V + E)$

Cuándo usarlo: MST. Requiere DSU. Mejor cuando el grafo se da como lista de aristas.

```
1 // Added -- not in notebook
2 // Requiere struct DSU definido arriba
3 ll kruskal(int n, vector<tuple<ll,int,int>> &edges){
4     sort(all(edges)); // ordenar por peso
5     DSU dsu(n);
6     ll cost = 0;
7     int count = 0;
```

```

8   for(auto &[w, u, v] : edges){
9       if(dsu.unite(u, v)){
10           cost += w;
11           count++;
12           if(count == n-1) break;
13       }
14   }
15   return cost; // si count < n-1, no es conexo
16 }
17 // --- USO ---
18 // vector<tuple<ll,int,int>> edges; // {peso, u, v}
19 // edges.push_back({5, 0, 1}); // arista 0-1 con peso 5
20 // ll mstCost = kruskal(n, edges);
21 // IMPORTANTE: tupla es {peso, u, v} (peso primero para sort).
22 // Requiere DSU definido arriba. Nodos 0-indexed.

```

Prim (Árbol de Expansión Mínima)

Tiempo: $O((V + E) \log V)$ **Espacio:** $O(V + E)$

Cuándo usarlo: MST. Mejor cuando el grafo es denso o se da como lista de adyacencia.

```

1 // Added -- not in notebook
2 ll prim(int n, vector<vector<pii>> &adj){
3     vector<bool> inMST(n, false);
4     // {peso, nodo}
5     priority_queue<pii, vector<pii>, greater<pii>> pq;
6     pq.push({0, 0});
7     ll cost = 0;
8     int count = 0;
9     while(!pq.empty() && count < n){
10         auto [w, v] = pq.top(); pq.pop();
11         if(inMST[v]) continue;
12         inMST[v] = true;
13         cost += w;
14         count++;
15         for(auto &[u, wt] : adj[v])
16             if(!inMST[u]) pq.push({wt, u});
17     }
18     return cost;
19 }
20 // --- USO ---
21 // adj: vector<vector<pii>>, pii = {nodo_vecino, peso}.
22 // ll mstCost = prim(n, adj); // MST desde nodo 0
23 // Empieza desde nodo 0. Si el grafo no es conexo, el MST es
   parcial.
24 // Misma adj que Dijkstra. Kruskal es mas facil si dan lista de
   aristas.

```

Componentes Fuertemente Conexas (Kosaraju)

Tiempo: $O(V + E)$ **Espacio:** $O(V + E)$

Cuándo usarlo: Encontrar SCCs en grafo dirigido. Útil para 2-SAT, condensación de grafos.

```

1 // Added -- not in notebook
2 vector<vi> adj, radj;
3 vector<bool> vis;
4 vi order, comp;
5
6 void dfs1(int v){
7     vis[v] = true;
8     for(int u : adj[v])
9         if(!vis[u]) dfs1(u);
10    order.push_back(v);
11 }
12
13 void dfs2(int v, int c){
14     comp[v] = c;
15     for(int u : radj[v])
16         if(comp[u] == -1) dfs2(u, c);
17 }
18
19 int kosaraju(int n){
20     vis.assign(n, false);
21     order.clear();
22     comp.assign(n, -1);
23     for(int i = 0; i < n; i++){
24         if(!vis[i]) dfs1(i);
25     }
26     int numSCC = 0;
27     for(int i = n-1; i >= 0; i--){
28         int v = order[i];
29         if(comp[v] == -1)
30             dfs2(v, numSCC++);
31     }
32 }

```

```

30 }
31 return numSCC;
32 }
33 // --- USO ---
34 // Llenar adj (grafo dirigido) y radj (grafo con aristas
   invertidas):
35 // adj[u].push_back(v); radj[v].push_back(u);
36 // adj y radj deben tener tamaño n.
37 // int numSCC = kosaraju(n);
38 // comp[v] = ID de la SCC a la que pertenece v (0-indexed).
39 // Nodos con mismo comp[v] estan en la misma componente.
40 // numSCC = cantidad total de componentes fuertemente conexas.

```

Puentes y Puntos de Articulación

Tiempo: $O(V + E)$ **Espacio:** $O(V + E)$

Cuándo usarlos: Encontrar aristas/nodos cuya eliminación desconecta el grafo.

```

1 // Added -- not in notebook
2 int timer_ba = 0;
3 vector<int> tin_ba, low;
4 vector<bool> vis_ba;
5 vector<pii> bridges;
6 vector<bool> isAP;
7
8 void dfs_bridge(int v, int p, vector<vi> &adj){
9     vis_ba[v] = true;
10    tin_ba[v] = low[v] = timer_ba++;
11    int children = 0;
12    for(int u : adj[v]){
13        if(u == p) continue;
14        if(vis_ba[u]){
15            low[v] = min(low[v], tin_ba[u]);
16        } else {
17            children++;
18            dfs_bridge(u, v, adj);
19            low[v] = min(low[v], low[u]);
20            // Puente
21            if(low[u] > tin_ba[v])
22                bridges.push_back({v, u});
23            // Punto de articulacion
24            if(p != -1 && low[u] >= tin_ba[v])
25                isAP[v] = true;
26        }
27    }
28    if(p == -1 && children > 1)
29        isAP[v] = true;
30 }
31
32 void findBridgesAP(int n, vector<vi> &adj){
33     timer_ba = 0;
34     tin_ba.assign(n, -1);
35     low.assign(n, -1);
36     vis_ba.assign(n, false);
37     isAP.assign(n, false);
38     bridges.clear();
39     for(int i = 0; i < n; i++){
40         if(!vis_ba[i]) dfs_bridge(i, -1, adj);
41     }
42     // --- USO ---
43     // adj = lista de adyacencia NO dirigida.
44     // findBridgesAP(n, adj);
45     // bridges = vector de pares {u,v} que son puentes (aristas
46     // cuya eliminacion desconecta el grafo).
47     // isAP[v] = true si v es punto de articulacion (nodo cuya
48     // eliminacion desconecta el grafo).

```

LCA (Ancestro Común Más Bajo — Binary Lifting)

Tiempo: Preproceso $O(n \log n)$, consulta $O(\log n)$ **Espacio:** $O(n \log n)$

Cuándo usarlo: Consultas de ancestro común en árboles, distancia en árboles.

Explicación: Precalcula para cada nodo sus antepasados a potencias de 2. Para consultar LCA, primero sube el más profundo hasta igualar niveles, luego sube a ambos juntos. La distancia entre dos nodos es $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$.

```

1 // Added -- not in notebook
2 const int LOGN = 20;

```

```

3 int up[200001][LOGN];
4 int depth[200001];
5
6 void dfs_lca(int v, int p, int d, vector<vi> &adj){
7     up[v][0] = p;
8     depth[v] = d;
9     for(int k = 1; k < LOGN; k++){
10         up[v][k] = up[up[v][k-1]][k-1];
11     }
12     for(int u : adj[v])
13         if(u != p) dfs_lca(u, v, d+1, adj);
14 }
15
16 int lca(int u, int v){
17     if(depth[u] < depth[v]) swap(u, v);
18     int diff = depth[u] - depth[v];
19     for(int k = 0; k < LOGN; k++){
20         if((diff >> k) & 1) u = up[u][k];
21     }
22     if(u == v) return u;
23     for(int k = LOGN-1; k >= 0; k--){
24         if(up[u][k] != up[v][k]){
25             u = up[u][k];
26             v = up[v][k];
27         }
28     }
29     return up[u][0];
30 }
31
32 int dist_tree(int u, int v){
33     return depth[u] + depth[v] - 2*depth[lca(u,v)];
34 }
35
36 // --- USO ---
37 // 1. Construir adj como arbol no dirigido.
38 // 2. dfs_lca(root, root, 0, adj); // preprocesar desde la raiz
39 // (root suele ser 0 o 1). Primer y segundo arg = root.
40 // 3. int a = lca(u, v); // ancestro comun mas bajo de u y v
41 // 4. int d = dist_tree(u,v); // distancia (aristas) entre u y v
42 // en el arbol
43 // LOGN=20 soporta hasta 2^20 = 1M nodos. Cambiar si n > 1M.

```

Heavy-Light Decomposition (HLD)

Tiempo: Preproceso $O(n)$, consulta/update en camino $O(\log^2 n)$ **Espacio:** $O(n)$

Cuándo usarlo: Consultas/updates sobre caminos en árbol: suma, máx, mín, etc. Requiere Segment Tree o BIT sobre el arreglo linealizado pos[].

Explicación: Descompone el árbol en cadenas pesadas (heavy paths). Cada nodo pertenece a exactamente una cadena. El arreglo pos[] asigna a cada nodo una posición tal que cada cadena es un segmento contiguo. Las consultas en camino saltan entre cadenas usando LCA implícito.

```

1 // Added
2 vector<int> parent, depth, heavy, head, pos;
3 int cur_pos;
4
5 int dfs_hld(int v, vector<vi> &adj){
6     int size = 1, max_c_size = 0;
7     for(int c : adj[v]){

```

```

8         if(c == parent[v]) continue;
9         parent[c] = v;
10        depth[c] = depth[v] + 1;
11        int c_size = dfs_hld(c, adj);
12        size += c_size;
13        if(c_size > max_c_size){
14            max_c_size = c_size;
15            heavy[v] = c;
16        }
17    }
18    return size;
19 }
20
21 void decompose(int v, int h, vector<vi> &adj){
22     head[v] = h; pos[v] = cur_pos++;
23     if(heavy[v] != -1)
24         decompose(heavy[v], h, adj);
25     for(int c : adj[v]){
26         if(c != parent[v] && c != heavy[v])
27             decompose(c, c, adj);
28     }
29 }
30
31 void initHLD(int n, int root, vector<vi> &adj){
32     parent.assign(n, -1);
33     depth.assign(n, 0);
34     heavy.assign(n, -1);
35     head.assign(n, 0);
36     pos.assign(n, 0);
37     cur_pos = 0;
38     dfs_hld(root, adj);
39     decompose(root, root, adj);
40 }
41
42 // Query en camino u->v (asume SegTree sobre pos[])
43 ll queryPath(int u, int v, SegTree &st){
44     ll res = 0; // identidad: 0 para suma, INF para min
45     while(head[u] != head[v]){
46         if(depth[head[u]] < depth[head[v]]) swap(u, v);
47         res += st.query(pos[head[u]], pos[u]);
48         u = parent[head[u]];
49     }
50     if(depth[u] > depth[v]) swap(u, v);
51     res += st.query(pos[u], pos[v]);
52     return res;
53 }
54
55 // Update en nodo (puntual)
56 void updateNode(int v, ll val, SegTree &st){
57     st.update(pos[v], val);
58 }
59
60 // --- USO ---
61 // initHLD(n, root, adj); // preprocesar
62 // Valores iniciales: for(i) st.update(pos[i], val[i]);
63 // queryPath(u, v, st) -> consulta en camino u-v
64 // Para update de rango en camino: similar a queryPath con update
65 // en st.
66 // Ajustar SegTree: suma, max, min, xor, etc.
67 // HLD + SegTree permite updates puntuales y consultas en camino.
68 // Con Lazy SegTree se pueden hacer updates de rango en camino.

```

Flujo en Redes y Matching

Max Flow (Dinic)

Tiempo: $O(V^2E)$ general, $O(E\sqrt{V})$ bipartito **Espacio:** $O(V + E)$

Cuándo usarlo: Flujo máximo. Acepta capacidades ll. Más rápido que Edmonds-Karp en ICPC.

Explicación: BFS construye grafos de niveles (aristas con $\text{cap} > 0$). DFS empuja flujo respetando niveles. Repite hasta que no hay camino $s \rightarrow t$. Complejidad práctica mucho mejor que la teórica.

```

1 // Added
2 struct Dinic {
3     struct Edge { int to, rev; ll cap; };
4     vector<vector<Edge>> g;
5     vector<int> level, ptr; int n;
6
7     Dinic(int n) : n(n), g(n), level(n), ptr(n) {}
8
9     void addEdge(int u, int v, ll cap){
10         g[u].push_back({v, (int)g[v].size(), cap});
11         g[v].push_back({u, (int)g[u].size() - 1, 0});
12     }
13
14     bool bfs(int s, int t){
15         fill(all(level), -1);
16         queue<int> q; q.push(s); level[s] = 0;
17         while(!q.empty()){
18             int v = q.front(); q.pop();
19             for(auto &e : g[v])
20                 if(e.cap > 0 && level[e.to] == -1){
21                     level[e.to] = level[v] + 1;
22                     q.push(e.to);
23                 }
24         }
25         return level[t] != -1;
26     }
27
28     ll dfs(int v, int t, ll flow){
29         if(v == t) return flow;
30         for(int &i = ptr[v]; i < sz(g[v]); i++){
31             Edge &e = g[v][i];
32             if(e.cap > 0 && level[e.to] == level[v] + 1){
33                 ll pushed = dfs(e.to, t, min(flow, e.cap));
34                 if(pushed > 0){
35                     e.cap -= pushed;
36                     g[e.to][e.rev].cap += pushed;
37                     return pushed;
38                 }
39             }
40         }
41         return 0;
42     }
43
44     ll maxFlow(int s, int t){
45         ll flow = 0;
46         while(bfs(s, t)){
47             fill(all(ptr), 0);
48             while(ll pushed = dfs(s, t, LLONG_MAX))
49                 flow += pushed;
50         }
51         return flow;
52     }
53 };
54 // --- USO ---
55 // Dinic dinic(n);
56 // dinic.addEdge(u,v,cap); // arista dirigida u->v con cap
57 // ll f = dinic.maxFlow(source, sink);
58 // Min-cut: nodos con level!=-1 vs level==1 tras ultimo bfs.
59 // Grafo no-dirigido: addEdge(u,v,c); addEdge(v,u,c);

```

```

60 // Flujo max bipartito con super-source/sink:
61 // fuente->izq con cap=1, der->sink con cap=1, izq->der con cap
62 // =1

```

Max Bipartite Matching (Hopcroft-Karp)

Tiempo: $O(E\sqrt{V})$ **Espacio:** $O(V + E)$

Cuándo usarlo: Matching máximo en bipartito. Más rápido que Dinic genérico para este caso.

Explicación: Alterna BFS (encuentra caminos aumentantes más cortos) con DFS (aplica todos los disjuntos en paralelo). La distancia crece cada iteración $\rightarrow O(\sqrt{V})$ fases.

```

1 // Added
2 struct HopcroftKarp {
3     int n, m; vector<vi> g;
4     vi pairU, pairV, dist;
5
6     HopcroftKarp(int n, int m) : n(n), m(m), g(n),
7         pairU(n, -1), pairV(m, -1), dist(n) {}
8
9     void addEdge(int u, int v){ g[u].push_back(v); }
10
11     bool bfs(){
12         queue<int> q;
13         for(int u = 0; u < n; u++){
14             if(pairU[u] == -1){ dist[u] = 0; q.push(u); }
15             else dist[u] = -1;
16         }
17         bool found = false;
18         while(!q.empty()){
19             int u = q.front(); q.pop();
20             for(int v : g[u]){
21                 if(pairV[v] == -1) found = true;
22                 else if(dist[pairV[v]] == -1){
23                     dist[pairV[v]] = dist[u] + 1;
24                     q.push(pairV[v]);
25                 }
26             }
27         }
28         return found;
29     }
30
31     bool dfs(int u){
32         for(int v : g[u]){
33             if(pairV[v] == -1 ||
34                 (dist[pairV[v]] == dist[u]+1 && dfs(pairV[v]))){
35                 pairU[u] = v; pairV[v] = u; return true;
36             }
37         }
38         dist[u] = -1; return false;
39     }
40
41     int maxMatching(){
42         int match = 0;
43         while(bfs()){
44             for(int u = 0; u < n; u++)
45                 if(pairU[u] == -1 && dfs(u)) match++;
46             return match;
47         }
48     };
49 // --- USO ---
50 // HopcroftKarp hk(nIzq, nDer);
51 // hk.addEdge(i, j); // arista de izq i a der j
52 // int maxMatch = hk.maxMatching();
53 // pairU[i] = nodo der emparejado (o -1 si libre).
54 // Min Vertex Cover (K\onig):
55 // izq en cover = {u | dist[u] == -1}
56 // der en cover = {v | dist[pairV[v]] != -1}

```

Flujo en Redes II

2-SAT

Tiempo: $O(V + E)$ **Espacio:** $O(V + E)$

Cuándo usarlo: Variables booleanas x_i , cláusulas $(a \vee b)$. Determinar si hay asignación que satisface todas. Trigger típico: $n \leq 10^5$.

Explicación: $(a \vee b) \equiv (\neg a \rightarrow b) \wedge (\neg b \rightarrow a)$. Construye grafo de implicaciones $2n$ nodos. Si x y $\neg x$ en misma SCC, insatisfactible. Asignación: x_i true si $\text{comp}[x_i] > \text{comp}[\neg x_i]$.

```
1 // Added -- usa Kosaraju de sección Grafos
2 // Mapeo: variable i positiva -> i, negada -> i+n
3 // adj y radj deben tener 2*n nodos.
4
5 int neg2(int x, int n){ return x < n ? x + n : x - n; }
6
7 void addClause(int a, int b, vector<vi>& adj, vector<vi>& radj){
8     adj[neg2(a,n)].push_back(b);
9     adj[neg2(b,n)].push_back(a);
10    radj[b].push_back(neg2(a,n));
11    radj[a].push_back(neg2(b,n));
12 }
13
14 // Retorna asignacion. Vacio si no existe.
15 vector<bool> solve2SAT(int n, vector<pii>& clauses){
16     int N = 2 * n;
17     adj.assign(N, {}); radj.assign(N, {});
18     for(auto [a, b] : clauses) addClause(a, b, adj, radj);
19     int nsc = kosaraju(N);
20     vector<bool> ans(n);
21     for(int i = 0; i < n; i++){
22         if(comp[i] == comp[i + n]) return {};
23         ans[i] = (comp[i] > comp[i + n]);
24     }
25     return ans;
26 }
27 // --- USO ---
28 // n = numero de variables. clauses = {{a,b}, ...}
29 // a positivo = i, negativo = i+n.
30 // Ej: (x0 or ~x1) -> {0, 1+n}
31 // Requiere Kosaraju definido (adj, radj, comp globales).
32 // Adaptar segun si Kosaraju usa variables globales o locales.
```

Min-Cost Max-Flow (Dijkstra + Potencial)

Tiempo: $O(F \cdot E \log V)$ **Espacio:** $O(V + E)$

Cuándo usarlo: Flujo con costo por unidad. Capacidades y costos en ll. Acepta costos negativos.

```
1 // Added
2 struct MCMF {
```

```
3     struct Edge { int to, rev; ll cap, cost; };
4     vector<vector<Edge>> g; int n;
5     MCMF(int n) : n(n), g(n) {}
6
7     void addEdge(int u, int v, ll cap, ll cost){
8         g[u].push_back({v, (int)g[v].size(), cap, cost});
9         g[v].push_back({u, (int)g[u].size()-1, 0, -cost});
10    }
11
12    pair<ll, ll> minCostFlow(int s, int t){
13        ll flow = 0, cost = 0;
14        vector<ll> dist(n), pot(n, 0);
15        vi parent(n), pedge(n);
16        while(true){
17            fill(all(dist), 1e18); dist[s] = 0;
18            priority_queue<pll, vector<pll>, greater<pll>> pq;
19            pq.push({0, s});
20            while(!pq.empty()){
21                auto [d, v] = pq.top(); pq.pop();
22                if(d > dist[v]) continue;
23                for(int i = 0; i < sz(g[v]); i++){
24                    Edge &e = g[v][i];
25                    ll nd = d + e.cost + pot[v] - pot[e.to];
26                    if(e.cap > 0 && nd < dist[e.to]){
27                        dist[e.to] = nd;
28                        parent[e.to] = v; pedge[e.to] = i;
29                        pq.push({nd, e.to});
30                    }
31                }
32            }
33            if(dist[t] == 1e18) break;
34            for(int i = 0; i < n; i++){
35                if(dist[i] < 1e18) pot[i] += dist[i];
36            }
37            ll push = 1e18;
38            for(int v = t; v != s; v = parent[v])
39                push = min(push, g[parent[v]][pedge[v]].cap);
40            for(int v = t; v != s; v = parent[v]){
41                Edge &e = g[parent[v]][pedge[v]];
42                e.cap -= push; g[v][e.rev].cap += push;
43                cost += push * e.cost;
44            }
45            flow += push;
46        }
47        return {flow, cost};
48    }
49 };
50 // --- USO ---
51 // MCMF mcmf(n);
52 // mcmf.addEdge(u,v,cap,cost);
53 // auto [flow, cost] = mcmf.minCostFlow(s, t);
54 // Usa potencial de Johnson para costos negativos.
```


Teoría de Números

Criba de Eratóstenes

Tiempo: $O(n \log \log n)$ **Espacio:** $O(n)$

Cuándo usarla: Obtener todos los primos hasta n . $n \leq 10^7$ cabe en memoria.

Ejemplo: Marca como compuesto todo múltiplo de cada primo encontrado, empezando desde i^2 . Cuidado con $n = 0$ ó 1 (no son primos). Para $n > 10^7$, considerar criba segmentada.

```
1 // Del notebook: clasicos/TeoriaDeNumeros/criba.cpp
2 vector<bool> criba(int n){
3     vector<bool> primo(n+1, true);
4     primo[0] = primo[1] = false;
5     for(int i = 2; i*i <= n; i++){
6         if(primo[i])
7             for(int j = i*i; j <= n; j += i)
8                 primo[j] = false;
9     }
10    return primo;
11 }
```

Aplicado en: **776B (Codeforces)** — Clasificar números usando la criba para separar primos de compuestos.

```
1 // vjudge/776B-CodeForces.cpp (extracto)
2 vector<bool> isPrime = criba(100001);
3 for(int i = 2; i <= k+1; i++){
4     if(isPrime[i]) cout << 1 << ' ';
5     else cout << 2 << ' ';
6 }
```

Factorización en Primos

Tiempo: $O(\sqrt{n})$ **Espacio:** $O(\log n)$

Cuándo usarla: Obtener factores primos de un número. Contar divisores, sumar divisores, MCD/MCM.

Ejemplo: Divide por 2, luego por impares hasta $\sqrt{n}$. Si queda $n > 1$, ese residuo es primo. Cuidado con $n = 1$ (no tiene factores primos).

```
1 // Del notebook: clasicos/TeoriaDeNumeros/factorizacion_primos.cpp
2 vector<lli> factorizar(lli n){
3     vector<lli> sol;
4     while(n % 2 == 0)
5         sol.push_back(2), n /= 2;
6     for(lli d = 3; d*d <= n; d += 2)
7         while(n % d == 0) sol.push_back(d), n /= d;
8     if(n > 1) sol.push_back(n);
9     return sol;
10 }
11
12 // Version con conteo de exponentes
13 unordered_map<lli, lli> factorizarMap(lli n){
14     unordered_map<lli, lli> sol;
15     while(n % 2 == 0) sol[2]++, n /= 2;
16     for(lli d = 3; d*d <= n; d += 2)
17         while(n % d == 0) sol[d]++, n /= d;
18     if(n > 1) sol[n]++;
19     return sol;
20 }
```

Aplicado en: **Product and GCD (CF)** — Dado $p = \prod a_i$, encontrar el GCD máximo posible distribuyendo factores.

```
1 // practica-cpc/product-and-gcd.cpp (extracto)
2 auto f = factorizarMap(p);
3 ll k = 1;
4 for(auto &[base, exp] : f)
5     k *= binpow(base, exp / n);
```

Exponenciación Modular Rápida

Tiempo: $O(\log b)$ **Espacio:** $O(1)$ iterativo, $O(\log b)$ recursivo

Cuándo usarla: Calcular $a^b \bmod m$ eficientemente. Base de inverso modular, combinatoria.

```
1 // Del notebook: clasicos/TeoriaDeNumeros/potencia_modulo.cpp
2 // Version recursiva
3 long long binpow(long long a, long long b, long long m){
4     if(b == 0) return 1;
5     if(b % 2) return binpow(a, b-1, m) * a % m;
6     long long r = binpow(a, b/2, m);
7     return r * r % m;
8 }
9
10 // Version iterativa (preferida, sin stack overflow)
11 // Added -- not in notebook
12 long long binpow_iter(long long a, long long b, long long m){
13     long long res = 1;
14     a %= m;
15     while(b > 0){
16         if(b & 1) res = res * a % m;
17         a = a * a % m;
18         b >>= 1;
19     }
20     return res;
21 }
```

Aplicado en: **The Last Digit (SPOJ)** — Calcular $a^b \bmod 10$ para obtener el último dígito.

GCD y LCM

Tiempo: $O(\log(\min(a, b)))$ **Espacio:** $O(1)$

Cuándo usarlos: Simplificar fracciones, encontrar períodos, problemas de divisibilidad.

```
1 // Added -- not in notebook
2 // C++17 tiene __gcd() y en <numeric>: gcd(), lcm()
3 ll gcd(ll a, ll b){ return b ? gcd(b, a%b) : a; }
4 ll lcm(ll a, ll b){ return a / gcd(a, b) * b; }
5 // CUIDADO: a/gcd primero para evitar overflow
6 // GCD de un arreglo:
7 // ll g = 0; for(ll x : v) g = gcd(g, x);
```

Euclides Extendido

Tiempo: $O(\log(\min(a, b)))$ **Espacio:** $O(\log(\min(a, b)))$

Cuándo usarlo: Encontrar x, y tales que $ax + by = \gcd(a, b)$. Base del inverso modular.

```
1 // Added -- not in notebook
2 ll extgcd(ll a, ll b, ll &x, ll &y){
3     if(b == 0){
4         x = 1; y = 0;
5         return a;
6     }
7     ll x1, y1;
8     ll g = extgcd(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - (a / b) * y1;
11    return g;
12 }
13 // --- USO ---
14 // ll x, y;
15 // ll g = extgcd(a, b, x, y); // encuentra x,y tal que a*x + b*y = g
16 // g = gcd(a,b). x e y pueden ser negativos.
17 // Ej: extgcd(35, 15, x, y) -> g=5, x=1, y=-2 (35*1 + 15*(-2) = 5)
18 // Sirve para resolver ecuaciones lineales ax + by = c
19 // (tiene solucion sii gcd(a,b) | c).
```

Inverso Modular

Tiempo: $O(\log m)$ **Espacio:** $O(1)$

Cuándo usarlo: Dividir bajo módulo: $a/b \bmod m = a \cdot b^{-1} \bmod m$. Necesario para combinatoria modular.

```
1 // Added -- not in notebook
2 // Metodo 1: Fermat (solo si m es primo)
3 ll modinv(ll a, ll m){
4     return binpow_iter(a, m-2, m);
5 }
6
7 // Metodo 2: Euclides extendido (m no necesita ser primo)
8 ll modinv_ext(ll a, ll m){
9     ll x, y;
```

```

10 ll g = extgcd(a, m, x, y);
11 if(g != 1) return -1; // no existe
12 return (x % m + m) % m;
13 }
14 // --- USO ---
15 // ll inv = modinv(3, 1e9+7); // inverso de 3 mod 1e9+7
16 // (a / b) % m = a % m * modinv(b, m) % m
17 // Ej: (6/3) mod 7 = 6 * modinv(3, 7) % 7 = 6*5%7 = 2
18 // modinv (Fermat): solo si m es PRIMO. Mas simple.
19 // modinv_ext: funciona con cualquier m, pero gcd(a,m) debe ser 1.
20 // Retorna -1 si el inverso no existe.

```

Combinatoria ($\binom{n}{r} \bmod p$)

Tiempo: Preproceso $O(n)$, consulta $O(1)$ **Espacio:** $O(n)$

Cuándo usarla: Contar combinaciones bajo módulo primo. Caminos en cuadrículas, distribuciones.

```

1 // Added -- not in notebook
2 const int MAXN = 2000001;
3 const ll MOD_C = 1e9 + 7;
4 ll fact[MAXN], inv_fact[MAXN];
5
6 void precompute(){
7     fact[0] = 1;
8     for(int i = 1; i < MAXN; i++)
9         fact[i] = fact[i-1] * i % MOD_C;
10    inv_fact[MAXN-1] = binpow_iter(fact[MAXN-1], MOD_C-2, MOD_C);
11    for(int i = MAXN-2; i >= 0; i--)
12        inv_fact[i] = inv_fact[i+1] * (i+1) % MOD_C;
13 }
14
15 ll nCr(int n, int r){
16     if(r < 0 || r > n) return 0;
17     return fact[n] % MOD_C * inv_fact[r] % MOD_C
18         * inv_fact[n-r] % MOD_C;
19 }
20 // --- USO ---
21 // precompute(); // llamar UNA VEZ al inicio del main
22 // ll c = nCr(10, 3); // C(10,3) mod 1e9+7 = 120
23 // fact[n] = n! mod p. inv_fact[n] = (n!)^{-1} mod p.
24 // Requiere binpow_iter. MOD_C DEBE ser primo.
25 // MAXN debe ser >= max n que uses + 1.

```

Teorema Chino del Resto (CRT)

Tiempo: $O(n \log M)$ **Espacio:** $O(1)$

Cuándo usarlo: Resolver sistema $x \equiv r_i \pmod{m_i}$ con m_i coprimos dos a dos.

```

1 // Added -- not in notebook
2 // Dados vectores r[] y m[] de tamaño n,
3 // encuentra x tal que x = r[i] (mod m[i])
4 // Requiere extgcd definido arriba.
5 pair<ll,ll> crt(vector<ll> &r, vector<ll> &m){
6     ll curR = r[0], curM = m[0];
7     for(int i = 1; i < (int)r.size(); i++){
8         ll x, y;
9         ll g = extgcd(curM, m[i], x, y);
10        if((r[i] - curR) % g != 0)
11            return {-1, -1}; // sin solucion
12        ll lcm_val = curM / g * m[i];
13        ll diff = (r[i] - curR) / g;
14        // Usar __int128 para evitar overflow
15        ll shift = (__int128)diff * x % (m[i]/g);
16        curR = curR + curM * shift;
17        curM = lcm_val;
18        curR = ((curR % curM) + curM) % curM;
19    }
20    return {curR, curM};
21 }
22 // --- USO ---
23 // Resolver: x = 2 (mod 3), x = 3 (mod 5), x = 2 (mod 7)
24 // vector<ll> r = {2, 3, 2}, m = {3, 5, 7};
25 // auto [x, lcm] = crt(r, m); // x=23, lcm=105
26 // Solucion: x + k*lcm para cualquier k >= 0.
27 // Si retorna {-1,-1}, no hay solucion.
28 // m[i] NO necesitan ser primos, pero si coprimos dos a dos.
29 // Requiere extgcd definido arriba.

```

SPF — Factorización en $O(\log n)$

Tiempo: Preproceso $O(n \log \log n)$, consulta $O(\log n)$

Espacio: $O(n)$

Cuándo usarlo: Factorizar muchos números $\leq n$. $n \leq 10^7$ en ICPC (≈ 10 MB). Para $n > 10^7$, usar Pollard.

Explicación: Precalcula el factor primo más pequeño (SPF) de cada número. Factoriza $x \leq n$ en $O(\log x)$ dividiendo por su SPF. Base para totient, mobius, número de divisores.

```

1 // Added
2 const int MAXS = 10000001;
3 int spf[MAXS];
4
5 void buildSPF(){
6     iota(spf, spf + MAXS, 0);
7     for(int i = 2; i * i < MAXS; i++)
8         if(spf[i] == i)
9             for(int j = i * i; j < MAXS; j += i)
10                if(spf[j] == j) spf[j] = i;
11 }
12
13 vector<int> factorize(int x){
14     vector<int> f;
15     while(x > 1){ f.push_back(spf[x]); x /= spf[x]; }
16     return f;
17 }
18
19 // Pares {primo, exponente}
20 vector<pii> factorizeExp(int x){
21     vector<pii> f;
22     while(x > 1){
23         int p = spf[x], cnt = 0;
24         while(spf[x] == p){ cnt++; x /= p; }
25         f.push_back({p, cnt});
26     }
27     return f;
28 }
29 // --- USO ---
30 // buildSPF(); auto f = factorize(84); // {2,2,3,7}
31 // MAXS=10^7 ocupa 40MB con int; usar short para 20MB.
32 // Con SPF puedes precomputar totient, mobius, etc.

```

Función φ de Euler (Totient)

Tiempo: $O(\sqrt{n})$ sin SPF, $O(\log n)$ con SPF **Espacio:** $O(1)$

Cuándo usarlo: Contar coprimos con n . $\sum_{d|n} \varphi(d) = n$. Útil para teorema de Euler: $a^{\varphi(m)} \equiv 1 \pmod{m}$ si $\gcd(a, m) = 1$.

```

1 // Added
2 ll phi(ll n){
3     ll res = n;
4     for(ll p = 2; p * p <= n; p++){
5         if(n % p == 0){
6             while(n % p == 0) n /= p;
7             res -= res / p;
8         }
9     }
10    if(n > 1) res -= res / n;
11    return res;
12 }
13
14 // O(log n) con SPF (requiere buildSPF)
15 ll phiSPF(int x){
16     ll res = x;
17     while(x > 1){
18         int p = spf[x]; res -= res / p;
19         while(spfx == p) x /= p;
20     }
21     return res;
22 }
23 // --- USO ---
24 // phi(12) = 4 (1,5,7,11 son coprimos con 12)

```

Miller-Rabin (Determinístico 64-bit)

Tiempo: $O(\log n)$ **Espacio:** $O(1)$

Cuándo usarlo: Test de primalidad para $n < 2^{64}$ ($\approx 1.8 \times 10^{19}$). NO usar para $n \geq 2^{64}$.


```

37     }
38     return true;
39 }
40 // --- USO ---
41 // isPrime(9999999999999999999LL) -> true
42 // Si n cabe en 32 bits, bastan bases {2,7,61}.

```

Cuándo usarlo: Factorizar números entre 10^7 y 10^{18} . Pocos números, no conviene SPF.

```

1 // Added -- Requiere isPrime, mulmod, gcd de arriba
2 ll pollardRho(ll n){
3     if(n % 2 == 0) return 2;
4     if(n % 3 == 0) return 3;
5     while(true){
6         ll c = rand() % (n - 1) + 1;
7         auto f = [&](ll x){ return (mulmod(x,x,n)+c) % n; };
8         ll x = 2, y = 2, d = 1;
9         while(d == 1){
10             x = f(x); y = f(f(y));
11             d = gcd(abs(x - y), n);
12         }
13         if(d != n) return d;
14     }
15 }
16
17 void factorizeBig(ll n, vector<ll> &f){
18     if(n == 1) return;
19     if(isPrime(n)){ f.push_back(n); return; }
20     ll d = pollardRho(n);
21     factorizeBig(d, f); factorizeBig(n/d, f);
22 }
23 // --- USO ---
24 // vector<ll> f; factorizeBig(1e18+3, f); sort(all(f));
25 // srand(time(0)) en main si quieres variabilidad.

```

Teoría de Juegos

Juego de Nim y Teorema de Sprague-Grundy

Tiempo: $O(n)$ (Nim puro) **Espacio:** $O(1)$

Cuándo usarla: Juegos combinatorios imparciales donde dos jugadores alternan turnos y pierde quien no puede mover. El XOR de los tamaños de las pilas determina el ganador.

Resultado clave: En Nim con pilas $a_1, a_2, \dots, a_n$, el primer jugador gana si y solo si $a_1 \oplus a_2 \oplus \dots \oplus a_n \neq 0$.

```
1 // Nim puro: determina si el primer jugador gana
2 bool nimGana(vector<int>& pilas){
3     int xorSum = 0;
4     for(int x : pilas) xorSum ^= x;
5     return xorSum != 0; // true = gana J1
6 }
```

Teorema de Sprague-Grundy

Todo juego combinatorio imparcial es equivalente a una pila de Nim de tamaño g (el **número de Grundy**).

- $g(\text{posición perdedora}) = 0$
- $g(p) = \text{mex}(\{g(q) : q \in \text{movimientos}(p)\})$
- **mex** (minimum excludant): menor entero ≥ 0 que no está en el conjunto.
- Para juegos compuestos de subjuegos independientes: $g = g_1 \oplus g_2 \oplus \dots \oplus g_k$.

```
1 // Calcula el numero de Grundy para posiciones 0..n
2 // moves: conjunto de movimientos permitidos
3 vector<int> Grundy(int n, vector<int>& moves){
4     vector<int> g(n+1, 0);
5     for(int i = 1; i <= n; i++){
6         set<int> alcanzables;
7         for(int m : moves)
8             if(i - m >= 0)
9                 alcanzables.insert(g[i - m]);
10        // mex
11        int mex = 0;
12        while(alcanzables.count(mex)) mex++;
13        g[i] = mex;
14    }
15    return g;
16 }
```

Ejemplo: Juego donde puedes quitar 1, 2 ó 3 piedras. $g(0) = 0, g(1) = 1, g(2) = 2, g(3) = 3, g(4) = 0, g(5) = 1$,

... Período 4. Gana J1 si $n \bmod 4 \neq 0$.

Juegos Clásicos

Nim con restricción (Bounded Nim)

Si cada turno puedes quitar entre 1 y k piedras de una pila de n : gana J1 si $n \bmod (k+1) \neq 0$.

Staircase Nim

Pilas en escalera $a_0, a_1, \dots, a_n$. En cada turno, mueves piedras de pila i a pila $i-1$. Gana J1 si el XOR de las pilas en índices **impares** es $\neq 0$.

```
1 bool staircaseNim(vector<int>& pilas){
2     int xorSum = 0;
3     for(int i = 1; i < (int)pilas.size(); i += 2)
4         xorSum ^= pilas[i];
5     return xorSum != 0;
6 }
```

Juego de Euclides

Dos números (a, b) con $a \geq b$. En cada turno, restas un múltiplo positivo de b a a (sin que quede negativo). Pierde quien no puede mover (cuando $b = 0$).

```
1 // Retorna true si el jugador actual gana
2 bool euclidGame(int a, int b){
3     if(b == 0) return false;
4     if(a < b) swap(a, b);
5     if(a % b == 0) return true;
6     if(a >= 2*b) return true;
7     return !euclidGame(b, a - b);
8 }
```

Referencia Rápida

Juego	Gana J1 si
Nim puro	$\bigoplus a_i \neq 0$
Quitar 1...k	$n \bmod (k+1) \neq 0$
Staircase Nim	$\bigoplus a_{\text{impar}} \neq 0$
Wythoff (a, b)	$(a, b) \neq (\lfloor k\phi \rfloor, \lfloor k\phi^2 \rfloor)$
Euclides	ver recursion arriba
General	$g(\text{pos}) \neq 0$ (S-G)

Donde $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (razón áurea).

Cadenas (Strings)

KMP (Knuth-Morris-Pratt)

Tiempo: $O(n + m)$ **Espacio:** $O(m)$

Cuándo usarlo: Búsqueda de patrón en texto. Encontrar todas las ocurrencias de P en T .

Explicación: Precalcula el array 'lps' (longest proper prefix which is also suffix). Luego recorre el texto usando dos índices. Cuando hay mismatch, se mueve el índice del patrón según 'lps'.

```
1 // Added -- not in notebook
2 vector<int> computeLPS(const string &p){
3     int m = p.size();
4     vector<int> lps(m, 0);
5     int len = 0, i = 1;
6     while(i < m){
7         if(p[i] == p[len]){
8             lps[i++] = ++len;
9         } else {
10             if(len) len = lps[len-1];
11             else lps[i++] = 0;
12         }
13     }
14     return lps;
15 }
16
17 vector<int> kmpSearch(const string &text, const string &pat){
18     int n = text.size(), m = pat.size();
19     vector<int> lps = computeLPS(pat);
20     vector<int> matches;
21     int i = 0, j = 0;
22     while(i < n){
23         if(text[i] == pat[j]){
24             i++; j++;
25         }
26         if(j == m){
27             matches.push_back(i - j);
28             j = lps[j-1];
29         } else if(i < n && text[i] != pat[j]){
30             if(j) j = lps[j-1];
31             else i++;
32         }
33     }
34     return matches;
35 }
36 // --- USO ---
37 // auto pos = kmpSearch("abcbabc", "abc");
38 // pos = {0, 3, 6} -> posiciones donde aparece "abc" en el texto.
39 // computeLPS(pat) solo se necesita internamente.
40 // lps[i] = largo del prefijo mas largo de pat que tambien es
41 // sufixo de pat[0..i]. Util solo para encontrar periodo del
42 // patron.
```

Z-Algorithm

Tiempo: $O(n)$ **Espacio:** $O(n)$

Cuándo usarlo: Alternativa a KMP. $z[i]$ = longitud del prefijo más largo de s que coincide empezando en $s[i]$.

```
1 // Added -- not in notebook
2 vector<int> zFunction(const string &s){
3     int n = s.size();
4     vector<int> z(n, 0);
5     int l = 0, r = 0;
6     for(int i = 1; i < n; i++){
7         if(i < r)
8             z[i] = min(r - i, z[i - l]);
9         while(i + z[i] < n && s[z[i]] == s[i + z[i]])
10             z[i]++;
11         if(i + z[i] > r){
12             l = i;
13             r = i + z[i];
14         }
15     }
16     return z;
17 }
18
19 // Buscar patron P en texto T:
20 // string s = P + "$" + T;
21 // auto z = zFunction(s);
```

```
22 // for(int i = P.size()+1; i < s.size(); i++)
23 //     if(z[i] == P.size()) // match en posicion i-P.size()-1
```

Hashing de Cadenas (Polynomial Rolling)

Tiempo: Preproceso $O(n)$, consulta subcadena $O(1)$ **Espacio:** $O(n)$

Cuándo usarlo: Comparar subcadenas en $O(1)$, encontrar subcadenas repetidas, Rabin-Karp.

Explicación: Calcula un hash para cada prefijo. El hash de $s[l..r]$ se obtiene como $(h[r+1] - h[l] * pw[r-l+1]) \bmod M$. Usa dos módulos grandes para evitar colisiones (doble hash).

```
1 // Added -- not in notebook
2 struct StringHash {
3     vector<ll> h, pw;
4     ll base, mod;
5
6     StringHash(const string &s, ll base = 131,
7                 ll mod = 1e9+7)
8         : base(base), mod(mod) {
9         int n = s.size();
10        h.resize(n+1, 0);
11        pw.resize(n+1, 1);
12        for(int i = 0; i < n; i++){
13            h[i+1] = (h[i] * base + s[i]) % mod;
14            pw[i+1] = pw[i] * base % mod;
15        }
16    }
17
18    // Hash de s[l..r] (0-indexed, inclusive)
19    ll get(int l, int r){
20        return (h[r+1] - h[l] * pw[r-l+1] % mod + mod*2) % mod;
21    }
22 };
23 // --- USO ---
24 // StringHash h(s); // preprocesar string s
25 // ll ha = h.get(0, 4); // hash de s[0..4] (0-indexed,
26 //                       inclusive)
27 // ll hb = h.get(5, 9); // hash de s[5..9]
28 // if(ha == hb) -> subcadenas probablemente iguales.
29 // Para reducir colisiones, usar doble hash:
30 // StringHash h1(s, 131, 1e9+7);
31 // StringHash h2(s, 137, 1e9+9);
32 // Comparar pares (h1.get(l,r), h2.get(l,r))
```

Suffix Array (Básico)

Tiempo: $O(n \log^2 n)$ **Espacio:** $O(n)$

Cuándo usarlo: Subcadena común más larga, número de subcadenas distintas, búsqueda binaria sobre sufijos.

```
1 // Added -- not in notebook
2 vector<int> suffixArray(const string &s){
3     int n = s.size();
4     vector<int> sa(n), rank_(n), tmp(n);
5     iota(sa.begin(), sa.end(), 0);
6     for(int i = 0; i < n; i++) rank_[i] = s[i];
7     for(int k = 1; k < n; k <= 1){
8         auto cmp = [&](int a, int b){
9             if(rank_[a] != rank_[b]) return rank_[a] < rank_[b];
10            int ra = a+k < n ? rank_[a+k] : -1;
11            int rb = b+k < n ? rank_[b+k] : -1;
12            return ra < rb;
13        };
14        sort(sa.begin(), sa.end(), cmp);
15        tmp[sa[0]] = 0;
16        for(int i = 1; i < n; i++)
17            tmp[sa[i]] = tmp[sa[i-1]] + cmp[sa[i-1], sa[i]];
18        rank_ = tmp;
19        if(rank_[sa[n-1]] == n-1) break;
20    }
21    return sa;
22 }
23
24 // LCP array (Kasai)
25 vector<int> buildLCP(const string &s, const vector<int> &sa){
26     int n = s.size();
```

```

27 vector<int> rank_(n), lcp(n-1, 0);
28 for(int i = 0; i < n; i++) rank_[sa[i]] = i;
29 int k = 0;
30 for(int i = 0; i < n; i++){
31     if(rank_[i] == n-1){ k = 0; continue; }
32     int j = sa[rank_[i]+1];
33     while(i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
34     lcp[rank_[i]] = k;
35     if(k) k--;
36 }
37 return lcp;
38 }
39 // --- USO ---
40 // auto sa = suffixArray(s); // sa[i] = posicion del i-esimo
41 // sufijo en orden lexicografico. Ej: s="banana",
42 // sa = {5,3,1,0,4,2} -> sufijos ordenados: "a","ana","anana",...
43 // auto lcp = buildLCP(s, sa); // lcp[i] = prefijo comun mas
44 // largo entre sa[i] y sa[i+1] (sufijos consecutivos en orden).
45 // Subcadenas distintas = n*(n+1)/2 - sum(lcp[i]).
46 // Subcadena comun mas larga de 2 strings: concatenar con
47 // separador s1+"$"+s2, construir SA+LCP, buscar max lcp[i]
48 // donde sa[i] y sa[i+1] pertenecen a strings distintos.

```

Manacher (Palíndromos en $O(n)$)

Tiempo: $O(n)$ **Espacio:** $O(n)$

Cuándo usarlo: Subcadena palíndroma más larga, contar palíndromos, chequeo rápido de si $s[l..r]$ es palíndromo.

Explicación: Transforma s insertando # entre caracteres. Calcula $d[i]$ = radio del palíndromo centrado en i . Palín-

dromo impar: letra real. Par: #. $d[i]$ es el número de caracteres reales a cada lado.

```

1 // Added -- palindromos en O(n)
2 vector<int> manacher(const string &s){
3     string t = "|";
4     for(char c : s){ t += c; t += '|'; }
5     int n = t.size();
6     vector<int> d(n);
7     for(int i = 0, l = 0, r = -1; i < n; i++){
8         int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
9         while(i - k >= 0 && i + k < n && t[i - k] == t[i + k])
10             k++;
11         d[i] = k--;
12         if(i + k > r){ l = i - k; r = i + k; }
13     }
14     return d;
15 }
16
17 // Chequeo rapido O(1) de si s[l..r] es palindromo:
18 // (requiere d del manacher)
19 bool isPal(int l, int r, vector<int> &d){
20     int center = l + r + 1; // indice en t
21     return d[center] > r - l;
22 }
23 // --- USO ---
24 // auto d = manacher(s);
25 // int maxLen = *max_element(all(d)) - 1;
26 // isPal(l, r, d) -> true si s[l..r] inclusive es palindromo.
27 // d[2*i] = radio en '|' (palindromos pares).
28 // d[2*i+1] = radio en caracter real (palindromos impares).

```

Programación Dinámica

Mochila 0/1 (Knapsack)

Tiempo: $O(n \cdot W)$ **Espacio:** $O(W)$

Cuándo usarla: Seleccionar subconjunto de objetos maximizando valor sin exceder capacidad W .

Explicación: 'dp[j]' = máximo valor con capacidad 'j'. Se itera sobre objetos y se actualiza de atrás hacia adelante para no reusar el mismo objeto.

```
1 // Added -- not in notebook
2 int knapsack01(int n, int W, vector<int> &w, vector<int> &v){
3     // dp[j] = maximo valor con capacidad j
4     vector<int> dp(W+1, 0);
5     for(int i = 0; i < n; i++){
6         for(int j = W; j >= w[i]; j--){
7             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
8         }
9     }
10    // CUIDADO: recorrer j de W a w[i] (no al revés)
11    // para no usar el mismo item dos veces.
12    // Para mochila ilimitada: j de w[i] a W.
```

Subsecuencia Común Más Larga (LCS)

Tiempo: $O(n \cdot m)$ **Espacio:** $O(n \cdot m)$ (o $O(m)$ sin reconstrucción)

Cuándo usarla: Similitud entre dos secuencias. Diff de archivos.

```
1 // Added -- not in notebook
2 int lcs(const string &a, const string &b){
3     int n = a.size(), m = b.size();
4     // Optimización de espacio: solo 2 filas
5     vector<vector<int>> dp(2, vector<int>(m+1, 0));
6     for(int i = 1; i <= n; i++){
7         int cur = i & 1, prev = 1 - cur;
8         for(int j = 1; j <= m; j++){
9             if(a[i-1] == b[j-1])
10                dp[cur][j] = dp[prev][j-1] + 1;
11            else
12                dp[cur][j] = max(dp[prev][j], dp[cur][j-1]);
13        }
14    }
15    return dp[n & 1][m];
16 }

17 // Con reconstrucción (espacio O(n*m)):
18 string lcs_str(const string &a, const string &b){
19     int n = a.size(), m = b.size();
20     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
21     for(int i = 1; i <= n; i++){
22         for(int j = 1; j <= m; j++){
23             if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
24             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
25         }
26     }
27     string res;
28     int i = n, j = m;
29     while(i > 0 && j > 0){
30         if(a[i-1] == b[j-1]){ res += a[i-1]; i--; j--; }
31         else if(dp[i-1][j] > dp[i][j-1]) i--;
32         else j--;
33     }
34     reverse(all(res));
35     return res;
36 }

37 // --- USO ---
38 // int len = lcs("abcde", "ace"); // len = 3 ("ace")
39 // string sub = lcs_str("abcde", "ace"); // sub = "ace"
40 // lcs() solo retorna el largo (espacio O(m)).
41 // lcs_str() retorna la subsecuencia real (espacio O(n*m)).
42 // Funciona con strings o vectores (cambiar tipo).
```

Subsecuencia Creciente Más Larga (LIS)

Tiempo: $O(n \log n)$ **Espacio:** $O(n)$

Cuándo usarla: Encontrar la subsecuencia estrictamente creciente más larga.

Explicación: Usa un vector dp que mantiene la menor

cola posible para una longitud dada. Para cada x, busca la primera posición donde x puede reemplazar usando lower_bound (para estrictamente creciente) o upper_bound (para no decreciente). La reconstrucción guarda padres.

```
1 // Added -- not in notebook
2 int lis(vector<int> &a){
3     vector<int> dp;
4     for(int x : a){
5         auto it = lower_bound(all(dp), x);
6         if(it == dp.end()) dp.push_back(x);
7         else *it = x;
8     }
9     return dp.size();
10 }

11 // Para no-decreciente: usar upper_bound en vez de lower_bound
12 // Para reconstruir la subsecuencia:
13 vector<int> lis_reconstruct(vector<int> &a){
14     int n = a.size();
15     vector<int> dp, pos, parent(n, -1);
16     vector<int> idx; // posiciones en dp
17     for(int i = 0; i < n; i++){
18         auto it = lower_bound(all(dp), a[i]);
19         int k = it - dp.begin();
20         if(it == dp.end()){
21             dp.push_back(a[i]);
22             idx.push_back(i);
23         } else {
24             *it = a[i];
25             idx[k] = i;
26         }
27         parent[i] = (k > 0 ? idx[k-1] : -1);
28     }
29     vector<int> res;
30     int cur = idx[dp.size()-1];
31     while(cur != -1){
32         res.push_back(a[cur]);
33         cur = parent[cur];
34     }
35     reverse(all(res));
36     return res;
37 }

38 // --- USO ---
39 // vector<int> a = {3,1,4,1,5,9,2,6};
40 // int len = lis(a); // len = 4 (ej: {1,4,5,9})
41 // auto seq = lis_reconstruct(a); // retorna la subsecuencia
42 // lower_bound -> estrictamente creciente.
43 // upper_bound -> no-decreciente (permite iguales).
```

Digit DP (Plantilla)

Tiempo: $O(D \times S \times 10)$ donde D = dígitos, S = estados
Espacio: $O(D \times S)$

Cuándo usarla: Contar números en $[L, R]$ que cumplen cierta propiedad digit a digit.

Explicación: La función recursiva 'solve(pos, sum, tight)' recorre los dígitos de izquierda a derecha. 'tight' indica si el prefijo actual es igual al límite superior. La memoria (memo) se guarda por 'pos', 'sum' (u otros estados) y 'tight'.

```
1 // Added -- not in notebook
2 // Ejemplo: contar numeros en [1, N] cuya
3 // suma de digitos = target
4 string num;
5 ll memo[20][200][2]; // [pos][sum][tight]
6
7 ll solve(int pos, int sum, bool tight){
8     if(pos == (int)num.size())
9         return (sum == 0) ? 1 : 0; // ajustar condicion
10    if(memo[pos][sum][tight] != -1)
11        return memo[pos][sum][tight];
12    int limit = tight ? (num[pos] - '0') : 9;
13    ll res = 0;
14    for(int d = 0; d <= limit; d++){
15        res += solve(pos + 1, sum + d,
16                    tight && (d == limit));
17    }
18    return memo[pos][sum][tight] = res;
```

```
19 }  
20  
21 // Uso: contar en [1, N]  
22 // num = to_string(N);  
23 // memset(memo, -1, sizeof(memo));
```

```
24 // ll ans = solve(0, target, true);  
25 //  
26 // Para [L, R]: f(R) - f(L-1)
```

Geometría

Punto, Producto Cruz y Producto Punto

Tiempo: $O(1)$ por operación **Espacio:** $O(1)$

Cuándo usarlo: Base de toda la geometría computacional.

```
1 // Added -- not in notebook
2 typedef long long T; // usar double si se necesita
3 struct P {
4     T x, y;
5     P(T x=0, T y=0) : x(x), y(y) {}
6     P operator+(P p) const { return {x+p.x, y+p.y}; }
7     P operator-(P p) const { return {x-p.x, y-p.y}; }
8     P operator*(T t) const { return {x*t, y*t}; }
9     T dot(P p) const { return x*p.x + y*p.y; }
10    T cross(P p) const { return x*p.y - y*p.x; }
11    T norm2() const { return x*x + y*y; }
12    bool operator<(P p) const {
13        return tie(x,y) < tie(p.x, p.y);
14    }
15    bool operator==(P p) const {
16        return x == p.x && y == p.y;
17    }
18 };
19
20 // Orientacion de 3 puntos: >0 izq, <0 der, =0 colineal
21 T orient(P a, P b, P c){
22     return (b-a).cross(c-a);
23 }
24
25 // Distancia al cuadrado entre dos puntos
26 T dist2(P a, P b){
27     return (a-b).norm2();
28 }
```

Envolvente Convexa (Convex Hull — Andrew's Monotone Chain)

Tiempo: $O(n \log n)$ **Espacio:** $O(n)$

Cuándo usarla: Perímetro mínimo, diámetro de conjunto de puntos, eliminar puntos interiores.

```
1 // Added -- not in notebook
2 vector<P> convexHull(vector<P> pts){
3     int n = pts.size();
4     if(n < 2) return pts;
5     sort(all(pts));
6     vector<P> hull;
7     // Parte inferior
8     for(auto &p : pts){
9         while(hull.size() >= 2 &&
10             orient(hull[hull.size()-2],
11                 hull[hull.size()-1], p) <= 0)
12             hull.pop_back();
13         hull.push_back(p);
14     }
15     // Parte superior
16     int lower_size = hull.size() + 1;
17     for(int i = n-2; i >= 0; i--){
18         while((int)hull.size() >= lower_size &&
19             orient(hull[hull.size()-2],
20                 hull[hull.size()-1], pts[i]) <= 0)
21             hull.pop_back();
22         hull.push_back(pts[i]);
23     }
```

```
23 }
24 hull.pop_back(); // quitar ultimo duplicado
25 return hull;
26 }
27
28 // Area del poligono (con signo, dividir entre 2)
29 T areaPoligono2(vector<P> &poly){
30     T area = 0;
31     int n = poly.size();
32     for(int i = 0; i < n; i++)
33         area += poly[i].cross(poly[(i+1) % n]);
34     return area; // area real = abs(area) / 2
35 }
36 // --- USO ---
37 // vector<P> pts = {{0,0},{1,0},{1,1},{0,1},{0.5,0.5}};
38 // auto hull = convexHull(pts); // vertices del hull en orden
39 // T a2 = areaPoligono2(hull); // area * 2 (con signo)
40 // double area = abs(a2) / 2.0; // area real
41 // orient(a,b,c): >0 si c esta a la izquierda de a->b,
42 // <0 derecha, =0 colineal. Base de casi toda geometria.
43 // dist2(a,b): distancia al cuadrado (evita sqrt).
```

Intersección de Líneas

Tiempo: $O(1)$ **Espacio:** $O(1)$

Cuándo usarla: Encontrar punto de cruce de dos líneas/segmentos.

```
1 // Added -- not in notebook
2 // Línea definida por dos puntos: P a, P b
3 // Retorna {true, punto} si se intersectan,
4 // {false, {0,0}} si son paralelas.
5 // Usa double para precision
6 struct Pd {
7     double x, y;
8     Pd(double x=0, double y=0) : x(x), y(y) {}
9     Pd operator+(Pd p){ return {x+p.x, y+p.y}; }
10    Pd operator-(Pd p){ return {x-p.x, y-p.y}; }
11    Pd operator*(double t){ return {x*t, y*t}; }
12    double cross(Pd p){ return x*p.y - y*p.x; }
13 };
14
15 pair<bool, Pd> lineIntersection(Pd a, Pd b, Pd c, Pd d){
16     Pd ab = b - a, cd = d - c, ac = c - a;
17     double denom = ab.cross(cd);
18     if(abs(denom) < 1e-9) return {false, {0,0}};
19     double t = ac.cross(cd) / denom;
20     return {true, a + ab * t};
21 }
22
23 // Para segmentos: verificar que t in [0,1] y
24 // u = ac.cross(ab) / denom tambien in [0,1]
25 bool segmentIntersect(Pd a, Pd b, Pd c, Pd d){
26     Pd ab = b-a, cd = d-c, ac = c-a;
27     double denom = ab.cross(cd);
28     if(abs(denom) < 1e-9) return false; // paralelas
29     double t = ac.cross(cd) / denom;
30     double u = ac.cross(ab) / denom;
31     return t >= -1e-9 && t <= 1+1e-9
32         && u >= -1e-9 && u <= 1+1e-9;
33 }
```


Técnicas de Búsqueda

Búsqueda Binaria sobre la Respuesta

Tiempo: $O(\log(\text{rango}) \times f(n))$ **Espacio:** $O(1)$

Cuándo usarla: “Encontrar el mínimo/máximo valor tal que se cumple cierta condición monótona.”

```
1 // Added -- not in notebook
2 // Plantilla: encontrar el minimo x en [lo, hi]
3 // tal que check(x) es true.
4 // check debe ser monotona: F F F ... T T T
5 ll binarySearchAnswer(ll lo, ll hi){
6     while(lo < hi){
7         ll mid = lo + (hi - lo) / 2;
8         if(check(mid)) hi = mid;
9         else lo = mid + 1;
10    }
11    return lo;
12 }
13
14 // Para reales (ej. distancia minima):
15 double bsReal(double lo, double hi){
16     for(int iter = 0; iter < 100; iter++){
17         double mid = (lo + hi) / 2;
18         if(check(mid)) hi = mid;
19         else lo = mid;
20     }
21     return lo;
22 }
```

Dos Punteros (Two Pointers)

Tiempo: $O(n)$ **Espacio:** $O(1)$

Cuándo usarla: Arreglo ordenado, pares con cierta suma, subarreglos con propiedad monótona.

```
1 // Added -- not in notebook
2 // Ejemplo: encontrar par con suma = target en arreglo ordenado
3 pair<int,int> twoSum(vector<int> &a, int target){
4     int l = 0, r = (int)a.size() - 1;
5     while(l < r){
6         int s = a[l] + a[r];
7         if(s == target) return {l, r};
8         if(s < target) l++;
9         else r--;
10    }
11    return {-1, -1};
12 }
```

Ventana Deslizante (Sliding Window)

Tiempo: $O(n)$ **Espacio:** $O(k)$ o $O(1)$

Cuándo usarla: Subarreglos de tamaño fijo o variable con restricciones sobre elementos.

Aplicado en: Subarray Distinct Values (CSES) — Contar subarreglos con a lo más k valores distintos usando ventana deslizante con mapa de frecuencias.

```
1 // Del notebook: practica-cpc/subarray-distinct-values.cpp
2 void solve(){
3     int n, k; cin >> n >> k;
4     int l = 0;
5     map<int,int> freq;
6     ll count = 0;
7     vector<int> a(n);
8     for(int &x : a) cin >> x;
9     for(int r = 0; r < n; r++){
10        freq[a[r]]++;
11        while((int)freq.size() > k){
12            freq[a[l]]--;
13            if(freq[a[l]] == 0) freq.erase(a[l]);
14            l++;
15        }
16        count += (r - l + 1);
17    }
18    cout << count;
19 }
```

```
1 // Plantilla generica de ventana deslizante
2 // Added -- not in notebook
3 // Ventana de tamaño fijo k:
```

```
4 // mantener estado al agregar a[r] y quitar a[r-k]
5 // Ventana variable (maximo tamaño cumpliendo condicion):
6 int l = 0;
7 for(int r = 0; r < n; r++){
8     // agregar a[r] al estado
9     while(/* estado invalido */)
10        // quitar a[l] del estado
11        l++;
12 }
13 // procesar ventana [l, r]
14 // ej: ans = max(ans, r - l + 1);
15 }
```

Stack Monotónico (NGE/NSE/PGE/PSE)

Tiempo: $O(n)$ **Espacio:** $O(n)$

Cuándo usarlo: Para cada elemento a_i , encontrar el siguiente/anterior elemento mayor/menor. Nomenclatura: NGE (Next Greater), NSE (Next Smaller), PGE (Previous Greater), PSE (Previous Smaller).

Explicación: Recorre el arreglo manteniendo un stack con elementos en orden creciente o decreciente según se busque mayor o menor. Cuando un nuevo elemento rompe la monotonía, el tope del stack encuentra su respuesta.

```
1 // Added
2 // NGE[i] = indice del sig. elemento > a[i] (o n si no hay)
3 vector<int> nextGreater(vi &a){
4     int n = a.size();
5     vi res(n, n); stack<int> st;
6     for(int i = 0; i < n; i++){
7         while(!st.empty() && a[st.top()] < a[i]){
8             res[st.top()] = i; st.pop();
9         }
10        st.push(i);
11    }
12    return res;
13 }
14
15 // PGE[i] = indice del ant. elemento > a[i] (o -1)
16 vector<int> prevGreater(vi &a){
17     int n = a.size();
18     vi res(n, -1); stack<int> st;
19     for(int i = 0; i < n; i++){
20         while(!st.empty() && a[st.top()] <= a[i]) st.pop();
21         if(!st.empty()) res[i] = st.top();
22         st.push(i);
23     }
24     return res;
25 }
26 // --- USO ---
27 // Para NSE: cambiar '<' por '>' en nextGreater -> nextSmaller
28 // Para PSE: cambiar '<=' por '>=' en prevGreater -> prevSmaller
29 // Aplicaciones: area maxima en histograma, contribucion de
30 // cada elemento como max/min en subarreglos, siguiente mayor.
31 // Area max en histograma: para cada i, area = a[i]*(NSE[i]-PSE[i]-1)
```

Mo's Algorithm

Tiempo: $O((n + q)\sqrt{n})$ **Espacio:** $O(n + q)$

Cuándo usarlo: Consultas en rango $[l, r]$ sin actualizaciones. $n, q \leq 10^5$. No sirve si hay updates.

Explicación: Ordena las consultas por bloque de l : l/B ascendente, y dentro del mismo bloque, r ascendente (o alternado para optimizar). Mantiene punteros L, R que se expanden/contraen al mover entre consultas.

```
1 // Added
2 int BLOCK;
3
4 struct Query { int l, r, idx; };
5
6 vector<ll> moAlgorithm(vi &a, vector<Query> &q){
7     int n = a.size(), q = qs.size();
8     BLOCK = max(1, (int)(n / sqrt(q)));
9     sort(all(qs), [&](Query &x, Query &y){
```

```

10     int bx = x.l / BLOCK, by = y.l / BLOCK;
11     if(bx != by) return bx < by;
12     return (bx & 1) ? x.r > y.r : x.r < y.r;
13 });
14 vector<ll> ans(q);
15 int L = 0, R = -1;
16 ll cur = 0;
17 auto add = [&](int pos){ /* cur += a[pos]; */ };
18 auto remove = [&](int pos){ /* cur -= a[pos]; */ };
19 for(auto &q : qs){
20     while(L > q.r) add(--L);
21     while(R < q.r) add(++R);
22     while(L < q.l) remove(L++);

```

```

23     while(R > q.r) remove(R--);
24     ans[q.idx] = cur;
25 }
26 return ans;
27 }
28 // --- USO ---
29 // vector<Query> qs; // l, r inclusive 0-based
30 // auto ans = moAlgorithm(a, qs);
31 // BLOCK = n/sqrt(q) es buena eleccion.
32 // Optimizacion: ordenar r alternado (odd-even sort).
33 // Implementar add() y remove() segun el problema.

```

Miscelánea

Recorrido del Caballo (Warnsdorff)

Tiempo: $O(n^2)$ heurístico **Espacio:** $O(n^2)$

Cuándo usarlo: Knight's tour en tablero $n \times n$. Funciona bien para $n \geq 5$.

```
1 // Del notebook: clasicos/off-topic/king-tour.cpp
2 int dx[] = {-2,-2,2,2,1,-1,1,-1};
3 int dy[] = {1,-1,1,-1,2,2,-2,-2};
4
5 bool isValid(int x, int y, vector<vector<int>> &tab){
6     int n = tab.size();
7     return x>=0 && x<n && y>=0 && y<n && tab[x][y]==-1;
8 }
9
10 int countMoves(int x, int y, vector<vector<int>> &tab){
11     int c = 0;
12     for(int k = 0; k < 8; k++){
13         if(isValid(x+dx[k], y+dy[k], tab)) c++;
14     }
15     return c;
16 }
17 // Heurística de Warnsdorff: elegir la casilla
18 // con menor numero de movimientos disponibles.
```

Operaciones de Bits Útiles

Tiempo: $O(1)$ **Espacio:** $O(1)$

Cuándo usarlas: Máscaras de bits, subconjuntos, potencias de 2.

```
1 // Added -- not in notebook
2 // Verificar si n es potencia de 2
3 bool isPow2(int n){ return n > 0 && (n & (n-1)) == 0; }
4 // Bit mas significativo
5 int msb(int n){ return 31 - __builtin_clz(n); }
6 // Contar bits encendidos
7 int popcount(int n){ return __builtin_popcount(n); }
8 // Para long long: __builtin_popcountll(n)
9 // Iterar sobre subconjuntos de una mascara m:
10 // for(int s = m; s > 0; s = (s-1) & m) { ... }
11 // Iterar sobre todos los subconjuntos de tamaño k de n bits:
12 // int s = (1<<k) - 1;
13 // while(s < (1<<n)){
14 //     // procesar s
15 //     int c = s & (-s), r = s + c;
16 //     s = ((r ^ s) >> 2) / c | r;
17 // }
```

Ordered Set (pb_ds)

Tiempo: $O(\log n)$ por operación **Espacio:** $O(n)$

Cuándo usarlo: Set con acceso por índice (k-ésimo elemento) y order_of_key.

```
1 // Added -- not in notebook
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
```

```
4 using namespace __gnu_pbds;
5 typedef tree<int, null_type, less<int>,
6     rb_tree_tag, tree_order_statistics_node_update>
7     ordered_set;
8
9 // ordered_set os;
10 // os.insert(x);
11 // os.find_by_order(k); // iterador al k-esimo (0-based)
12 // os.order_of_key(x); // cuantos elementos < x
13 // Para multiset: usar pair<int,int> con (valor, id_unico)
```

Trucos STL Frecuentes

```
1 // Added -- not in notebook
2 // Eliminar duplicados de vector ordenado:
3 sort(all(v));
4 v.erase(unique(all(v)), v.end());
5
6 // Rotar vector (shift izquierdo por k):
7 rotate(v.begin(), v.begin()+k, v.end());
8
9 // Acumular:
10 ll sum = accumulate(all(v), 0LL);
11
12 // Partial sums (prefix sums):
13 vector<ll> ps(n+1, 0);
14 for(int i = 0; i < n; i++) ps[i+1] = ps[i] + a[i];
15 // Suma [l, r] = ps[r+1] - ps[l]
16
17 // lower_bound / upper_bound en set:
18 auto it = s.lower_bound(x); // >= x
19 auto it2 = s.upper_bound(x); // > x
20
21 // max_element / min_element:
22 int mx = *max_element(all(v));
```

Sumas de Prefijo 2D

Tiempo: Preproceso $O(nm)$, consulta $O(1)$ **Espacio:** $O(nm)$

Cuándo usarla: Suma en sub-rectángulo de una matriz.

```
1 // Added -- not in notebook
2 // Dado int a[N][M]
3 // Construir:
4 ll ps[N+1][M+1]; // todo inicializado a 0
5 for(int i = 1; i <= n; i++){
6     for(int j = 1; j <= m; j++){
7         ps[i][j] = a[i-1][j-1] + ps[i-1][j]
8             + ps[i][j-1] - ps[i-1][j-1];
9     }
10 }
11 // Consulta suma en rectangulo (r1,c1) a (r2,c2) 1-based:
12 ll query2D(int r1, int c1, int r2, int c2){
13     return ps[r2][c2] - ps[r1-1][c2]
14         - ps[r2][c1-1] + ps[r1-1][c1-1];
15 }
```

Álgebra Lineal

Eliminación Gaussiana (módulo)

Tiempo: $O(n^3)$ **Espacio:** $O(n^2)$

Cuándo usarlo: Resolver sistema lineal $Ax = b \pmod{m}$ con m primo. Rango de matriz, determinante, inversa.

Explicación: Escalonamiento fila por fila. Para cada columna, busca pivote no nulo. Si no hay, continúa (rango $< n$). Luego sustitución hacia atrás.

```
1 // Added -- módulo primo
2 ll mod = 1e9 + 7;
3
4 ll gauss(vector<vector<ll>> a, vector<ll> &b, vector<ll> &x){
5     int n = a.size();
6     for(int col = 0, row = 0; col < n && row < n; col++){
7         int sel = row;
8         for(int i = row; i < n; i++)
9             if(abs(a[i][col]) > abs(a[sel][col])) sel = i;
10        if(a[sel][col] == 0) continue;
11        swap(a[sel], a[row]); swap(b[sel], b[row]);
12        ll inv = modinv(a[row][col], mod);
13        for(int j = col; j < n; j++)
14            a[row][j] = a[row][j] * inv % mod;
15        b[row] = b[row] * inv % mod;
16        for(int i = 0; i < n; i++){
17            if(i == row) continue;
18            ll f = a[i][col];
19            if(f == 0) continue;
20            for(int j = col; j < n; j++)
21                a[i][j] = (a[i][j] - f * a[row][j]) % mod;
22            b[i] = (b[i] - f * b[row]) % mod;
23        }
24        row++;
25    }
26    x.assign(n, 0);
27    for(int i = 0; i < n; i++)
28        x[i] = (b[i] + mod) % mod;
29    return 0; // 0 = sol unica (con mod)
30 }
31 // --- USO ---
32 // a = matriz n x n, b = terminos independientes.
33 // Requiere modinv definido (inverso modular).
34 // Si el sistema es singular (inf o 0 soluciones), chequear.
35 // Para doubles: quitar módulo, usar abs(a[i][col])>eps.
```

Exponenciación de Matrices

Tiempo: $O(k^3 \log n)$ **Espacio:** $O(k^2)$

Cuándo usarlo: Recurrencias lineales ($n \leq 10^{18}$), conteo de caminos en grafos, Fibonacci rápido.

Explicación: Escribe la recurrencia como multiplicación matriz $\times$ vector. Eleva la matriz a n con exponenciación binaria.

```
1 // Added -- multiplicacion modulo
2 using Mat = vector<vector<ll>>;
3
4 Mat matMul(const Mat &A, const Mat &B, ll mod){
5     int n = A.size(), m = B[0].size(), p = A[0].size();
6     Mat C(n, vector<ll>(m, 0));
7     for(int i = 0; i < n; i++)
8         for(int k = 0; k < p; k++)
9             if(A[i][k])
10                for(int j = 0; j < m; j++)
11                    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod;
12    return C;
13 }
14
15 Mat matPow(Mat A, ll e, ll mod){
16     int n = A.size();
17     Mat res(n, vector<ll>(n, 0));
18     for(int i = 0; i < n; i++) res[i][i] = 1;
19     while(e){
20         if(e & 1) res = matMul(res, A, mod);
21         A = matMul(A, A, mod);
22         e >>= 1;
23     }
24     return res;
25 }
26 // --- USO ---
```

```
27 // Fibonacci: [[F_{n+1}], [F_n]] = [[1,1],[1,0]]^n * [[F_1],[F_0]]
28 // Mat fib = {{1,1},{1,0}};
29 // Mat res = matPow(fib, n, MOD);
30 // F_n = res[0][1] (si F_0=0, F_1=1)
```

FFT (Fast Fourier Transform)

Tiempo: $O(n \log n)$ **Espacio:** $O(n)$

Cuándo usarlo: Multiplicar polinomios con coeficientes reales. Convolución. $n + m \leq 10^6$. Para enteros exactos usar NTT.

Explicación: Evalúa dos polinomios en raíces de la unidad, multiplica punto a punto, interpola inversa. El resultado es la convolución. Redondear a entero si los coeficientes son enteros.

```
1 // Added
2 using cd = complex<double>;
3 const double PI = acos(-1);
4
5 void fft(vector<cd> &a, bool invert){
6     int n = a.size();
7     for(int i = 1, j = 0; i < n; i++){
8         int bit = n >> 1;
9         for(; j & bit; bit >>= 1) j ^= bit;
10        j ^= bit;
11        if(i < j) swap(a[i], a[j]);
12    }
13    for(int len = 2; len <= n; len <= 1){
14        double ang = 2 * PI / len * (invert ? -1 : 1);
15        cd wlen(cos(ang), sin(ang));
16        for(int i = 0; i < n; i += len){
17            cd w(1);
18            for(int j = 0; j < len/2; j++){
19                cd u = a[i+j], v = a[i+j+len/2] * w;
20                a[i+j] = u + v;
21                a[i+j+len/2] = u - v;
22                w *= wlen;
23            }
24        }
25    }
26    if(invert) for(cd &x : a) x /= n;
27 }
28
29 vector<ll> multiply(vector<ll> &a, vector<ll> &b){
30     vector<cd> fa(all(a)), fb(all(b));
31     int n = 1;
32     while(n < sz(a) + sz(b)) n <= 1;
33     fa.resize(n); fb.resize(n);
34     fft(fa, false); fft(fb, false);
35     for(int i = 0; i < n; i++) fa[i] *= fb[i];
36     fft(fa, true);
37     vector<ll> res(n);
38     for(int i = 0; i < n; i++) res[i] = round(fa[i].real());
39     return res;
40 }
41 // --- USO ---
42 // auto c = multiply(a, b); // c[k] = sum_{i+j=k} a[i]*b[j]
43 // Redondear con round(). Complejidad: O(n log n), n = potencia de 2.
```

NTT (Number Theoretic Transform)

Tiempo: $O(n \log n)$ **Espacio:** $O(n)$

Cuándo usarlo: Convolución con coeficientes enteros módulo p . Preferido sobre FFT en ICPC: sin errores de redondeo.

Explicación: Misma estructura que FFT pero con raíces de la unidad en Z_p . Requiere $p = c \cdot 2^k + 1$ primo. MOD = 998244353 ($119 \cdot 2^{23} + 1$) soporta $n \leq 2^{23}$.

```
1 // Added
2 const ll MOD_NTT = 998244353; // 119*2^23+1, raiz=3
3 const ll ROOT = 3;
4
5 ll ntt_pow(ll a, ll d, ll m){
6     ll res = 1;
7     while(d){
```

```

8     if(d & 1) res = res * a % m;
9     a = a * a % m; d >>= 1;
10 }
11 return res;
12 }
13
14 void ntt(vector<ll> &a, bool invert){
15     int n = a.size();
16     for(int i = 1, j = 0; i < n; i++){
17         int bit = n >> 1;
18         for(; j & bit; bit >>= 1) j ^= bit;
19         j ^= bit;
20         if(i < j) swap(a[i], a[j]);
21     }
22     for(int len = 2; len <= n; len <= 1){
23         ll wlen = ntt_pow(ROOT, (MOD_NTT-1)/len, MOD_NTT);
24         if(invert) wlen = ntt_pow(wlen, MOD_NTT-2, MOD_NTT);
25         for(int i = 0; i < n; i += len){
26             ll w = 1;
27             for(int j = 0; j < len/2; j++){
28                 ll u = a[i+j], v = a[i+j+len/2] * w % MOD_NTT;
29                 a[i+j] = (u + v) % MOD_NTT;
30                 a[i+j+len/2] = (u - v + MOD_NTT) % MOD_NTT;
31                 w = w * wlen % MOD_NTT;

```

```

32     }
33 }
34 }
35 if(invert){
36     ll inv_n = ntt_pow(n, MOD_NTT-2, MOD_NTT);
37     for(ll &x : a) x = x * inv_n % MOD_NTT;
38 }
39 }
40
41 vector<ll> multiplyNTT(vector<ll> a, vector<ll> b){
42     int n = 1;
43     while(n < sz(a) + sz(b)) n <= 1;
44     a.resize(n); b.resize(n);
45     ntt(a, false); ntt(b, false);
46     for(int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD_NTT;
47     ntt(a, true);
48     return a;
49 }
50 // --- USO ---
51 // auto c = multiplyNTT(a, b); // c[k] = sum a[i]*b[j] mod MOD
52 // MOD=998244353 soporta n<=2^23. O(n log n).
53 // Para otro primo p: usar raiz primitiva de p con orden >= n.

```

Scripts Útiles (Compilar y Probar)

Compilación Estándar (C++17)

Cuándo usarlo: Para compilar la solución asegurando el uso del estándar C++17 y activando advertencias de código comunes. Útil para verificar la sintaxis antes de ejecutar el código contra un caso de prueba.

```
1 # Compilar archivo sol.cpp y generar el ejecutable sol.exe (
  Windows) o sol (Linux/Mac)
2 g++ -std=c++17 -O2 -Wall -Wextra -o sol sol.cpp
```

Compilación y Ejecución Directa (CMD/Bash)

Cuándo usarlo: Para automatizar la prueba de la solución, redirigiendo un archivo de texto con las entradas directamente al programa. *Nota.* Requiere usar el Command Prompt en Windows o Bash en Linux/Mac; no funciona con PowerShell.

```
1 # Compila y, si es exitoso, ejecuta pasando entrada.txt, es
  forzoso tener un freopen("archivo.txt", "r", stdin)
2 g++ -std=c++17 -O2 -Wall -Wextra -o sol sol.cpp ; .\sol.exe
```

Redirección de I/O desde el Código

Cuándo usarlo: Cuando se trabaja en entornos donde redireccionar desde la terminal es complicado (como PowerShell) o para no tener que teclear el comando largo cada vez que se prueba. *Importante:* Comentar o eliminar esta línea antes de enviar el código al juez en línea (a menos que el problema explícitamente pida leer de archivo).

```
1 int main(){
2     fast;
3     // Agrega esto temporalmente para leer desde "entrada.txt"
4     freopen("entrada.txt", "r", stdin);
5
6     int t = 1;
7     // cin >> t;
8     while(t--) solve();
9     return 0;
10 }
```

Patrones Rápidos

Tabla de referencia rápida: dado el tipo de problema, qué algoritmo aplicar.

Señal del problema	Algoritmo / Técnica	Complejidad
Camino más corto, sin peso	BFS	$O(V + E)$
Camino más corto, pesos ≥ 0	Dijkstra	$O((V + E) \log V)$
Camino más corto, pesos negativos	Bellman-Ford	$O(VE)$
Camino más corto, pesos 0 ó 1	0-1 BFS	$O(V + E)$
Distancias entre todos los pares, $V \leq 500$	Floyd-Warshall	$O(V^3)$
Árbol de expansión mínima	Kruskal (con DSU) o Prim	$O(E \log E)$
Ordenar tareas con dependencias	Toposort (Kahn / DFS)	$O(V + E)$
Componentes fuertemente conexas	Kosaraju / Tarjan	$O(V + E)$
Puentes / puntos de articulación	DFS con tin/low	$O(V + E)$
Ancestro común en árbol	LCA (binary lifting)	$O(\log n)$ consulta
Componentes conexas dinámicas	DSU / Union-Find	$O(\alpha(n))$
Consulta mín/máx en rango estático	Sparse Table	$O(1)$ consulta
Suma de rango con actualización puntual	BIT / Fenwick Tree	$O(\log n)$
Rango con actualización de rango	Segment Tree + Lazy	$O(\log n)$
Siguiente/anterior mayor/menor	Stack monotónico (NGE/NSE/PGE/PSE)	$O(n)$
Área máxima en histograma	PSE + NSE + contribución	$O(n)$
Suma de (max-min) de todos los subarreglos	Contribución con NGE/PGE/NSE/PSE	$O(n)$
Búsqueda de patrón en texto	KMP / Z-algorithm	$O(n + m)$
Comparar subcadenas en $O(1)$	Hashing polinomial	$O(n)$ preproceso
Subcadena común más larga	Suffix Array + LCP	$O(n \log^2 n)$
Prefijos de un diccionario	Trie	$O(s)$
Todos los primos hasta n	Criba de Eratóstenes	$O(n \log \log n)$
$a^b \bmod m$	Exponenciación binaria	$O(\log b)$
Dividir bajo módulo	Inverso modular (Fermat/ExtGCD)	$O(\log m)$
$\binom{n}{r} \bmod p$	Factorial + inverso precalculado	$O(1)$ consulta
Sistema de congruencias	Teorema Chino del Resto	$O(n \log M)$
Mochila (subconjunto óptimo)	DP Knapsack 0/1	$O(nW)$
Subsecuencia común más larga	DP LCS	$O(nm)$
Subsecuencia creciente más larga	LIS con búsqueda binaria	$O(n \log n)$
Contar números con propiedad en $[L, R]$	Digit DP	$O(D \times S \times 10)$
Mínimo/máximo valor que cumple condición	Búsqueda binaria sobre la respuesta	$O(\log R \times f)$
Subarreglos con $\leq k$ distintos	Ventana deslizante + mapa	$O(n)$
Par con suma = t en arreglo ordenado	Dos punteros	$O(n)$
Perímetro mínimo / puntos extremos	Convex Hull	$O(n \log n)$
Intersección de líneas	Producto cruz + parámetro t	$O(1)$
K-ésimo elemento / rango	Ordered set (pb_ds)	$O(\log n)$
Fuerza bruta sobre permutaciones ($n \leq 10$)	next_permutation	$O(n!)$
Overflow en multiplicación	__int128 o mulmod	$O(1)$